

Byzantine-Tolerant Decentralized Proximal Gradient Methods

Gilbert Paul Neuner

May 15, 2026

1 Introduction

Federated learning (FL) a machine learning technique in which multiple clients collaborate to train a model while keeping their local datasets private. Often, individual clients lack sufficient data to identify patterns. However, centralizing data can be infeasible due to privacy regulations. For example, the General Data Protection Regulation (GDPR) regulates data exchange across mobile devices, and the Health Insurance Portability and Accountability Act of 1996 (HIPAA) regulates the exchange of patient data between healthcare institutions.

FL is exemplified by FedAvg [MMR16], which Google uses for next-word prediction on Android Gboard. In a round of FedAvg, clients communicate with a server. First, the server broadcasts to clients the shared global model. Then, clients obtain distinct local models by performing a step of gradient descent, based on their local data and the global model. Then, clients send their local models to the server. Finally, the server obtains the next global model by aggregating, that is, taking a weighted average, of local models.

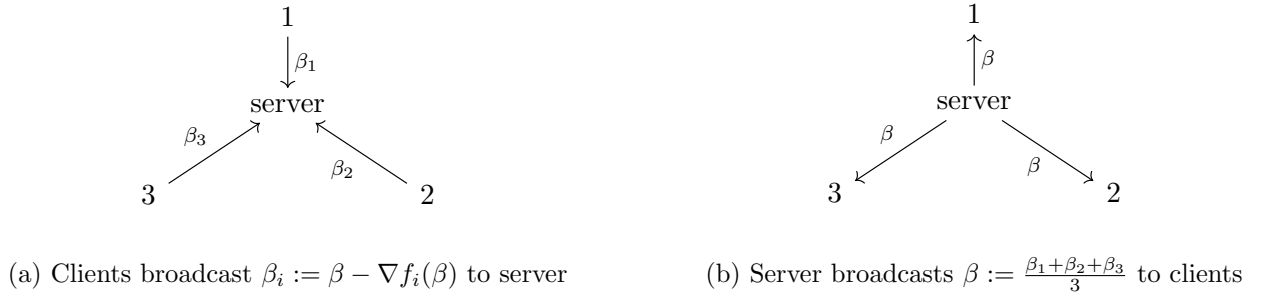


Figure 1: A round of FedAvg

Figure 1 gives a simplified depiction of a round of FedAvg. The communication graph of FedAvg is called centralized, or star-shaped, and FedAvg is an instance of centralized federated learning (CFL). By contrast, decentralized federated learning (DFL) is a type of FL method in which the shape of the communication graph is arbitrary, and there is no server. Clients instead communicate with other clients, and handle aggregation themselves.

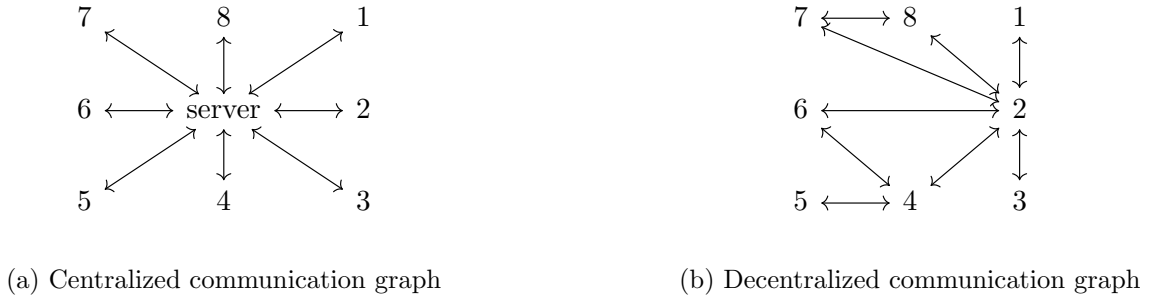


Figure 2: Centralized vs. decentralized communication graph

FL is susceptible to Byzantine failures (named after the Byzantine generals problem [LSP82]),

in which Byzantine clients broadcast arbitrary vectors instead of their true model. Byzantine clients might intentionally broadcast falsified models to prevent convergence, or they might be well-intentioned but unaware that their local data is poisoned.

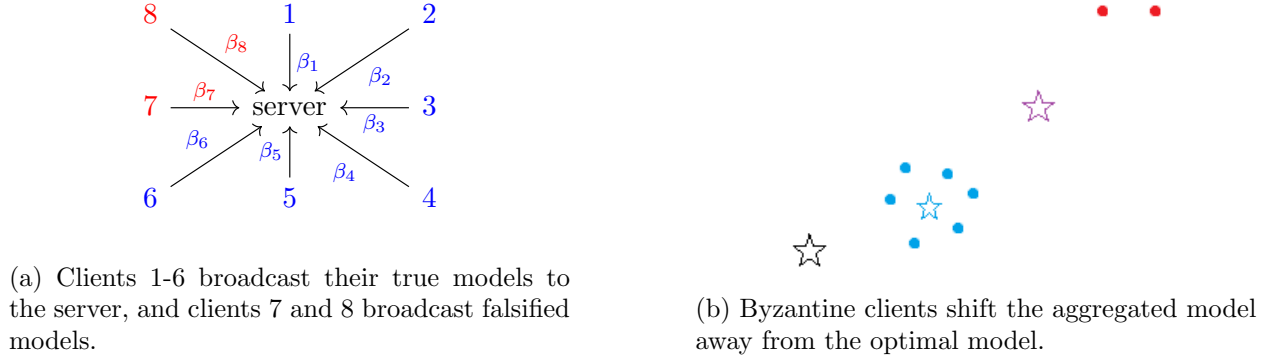


Figure 3: Example of Byzantine failure

Figure 3 gives a simple example of Byzantine failures. Clients 1-6 broadcast their true models to the server, and clients 7 and 8 broadcast falsified models. The optimal model is located at the black star, honest local models are located at the blue dots, and Byzantine local models are located at the red dots. Without Byzantine clients, the aggregated model would be located at the blue star. With Byzantine clients, the aggregated model is located at the purple star, further away from the optimal model.

1.1 Main Contribution

Our main contribution is a Byzantine-tolerant DFL method characterized by the following features:

- We give six versions of the main algorithm, each corresponding to a “model update sequence”. These model update sequences are the only ones satisfying specific constraints on communication cost and computational cost. To the best of our knowledge, two of these sequences are new, and one has only been used in CFL methods.
- Prior to aggregation, the magnitudes of the input vectors are “clipped”, protecting against magnitude attacks.
- During aggregation, clients weight the contributions of the neighbors based on “trust scores”, which quantify how suspiciously neighbors have behaved over the every round of the algorithm.

While the algorithm and main theory is applicable to convex, differentiable objective functions in general, in our numerical experiments we focus on the specific case of a LASSO objective function. The LASSO objective function is not differentiable, and accounting for the L^1 penalty term requires slight modification to the algorithm and convergence proof.

1.2 Roadmap

In section 2, further background on FL is given. In section 3, further background on Byzantine failures is given. In section 4, we motivate and state the main algorithm. In section 5, we restate

the main algorithm, which is a gradient method, as a proximal gradient method. In section 6, we give the main convergence result. In section 7, we give the results of multiple numerical experiments assessing the performance of the main algorithm on a LASSO problem. In section 8, we discuss future research plans.

1.3 Notation

- $[K]$: set of positive integers up to K , $\{1, \dots, K\}$.
- $\mathbb{N}$, $\mathbb{R}$, $\mathbb{R}^n$, $\mathbb{R}^{m \times n}$: the set of natural numbers, the set of real numbers, n -dimensional real vectors, $m \times n$ real matrices, respectively.
- ∇f : gradient of f .
- $\|\cdot\|$: L^2 norm.
- $\|\cdot\|_1$: L^1 norm.
- $\mathbb{E}$: expectation.
- $\langle x, y \rangle$: inner product $x^\top y$.
- $|A|$: for a set A , the cardinality of A .
- $\mathbf{1}\{\cdot\}$: indicator function.
- 0_p : vector of p zeros.
- I_p : identity matrix of dimension p .
- $N_p(\cdot, \cdot)$: p -dimensional normal distribution.
- $\text{tr}(\cdot)$: trace of a matrix.
- $X \perp Y \mid Z$: X and Y are conditionally independent given Z .
- $[\cdot]_{-i}$, $[\cdot]_{\cdot\ell}$, $[\cdot]_{\cdot-\ell}$: all but the i -th component of a vector, the ℓ -th column of a matrix, all but the ℓ -th column of a matrix, respectively.

2 Federated Learning

In **federated learning (FL)**, there are K **clients**, labeled by the members of the index set $[K] = \{1, \dots, K\}$. Each client $i \in [K]$ possesses a **local dataset** $D_i = \{(x_{i\ell}, y_{i\ell})\}_{\ell=1}^{n_i}$ where **feature vectors** $x_{i\ell} \in \mathbb{R}^p$ correspond to **outcomes** $y_{i\ell} \in \mathbb{R}$. Local datasets are **private**, i.e., only client i can access D_i . The i -th **local objective function** is defined by

$$f_i(\beta) = \frac{1}{n_i} \sum_{\ell=1}^{n_i} \phi(\beta; x_{i\ell}, y_{i\ell}),$$

where the parameter vector $\beta \in \mathbb{R}^p$ is referred to as a **model**, ϕ is a data point specific loss function, and f_i describes how well a model conforms to client i 's local data. The goal of FL is to minimize the **global objective function**

$$f(\beta) = \frac{1}{K} \sum_{i=1}^K f_i(\beta)$$

while preserving the privacy of local datasets. In this exam, we additionally adopt the following assumptions and notations:

- We consider algorithms which are synchronous and parallel. In a **synchronous** algorithm, computation is divided into **rounds**, and the next round starts when all clients are finished with the previous round. In a **parallel** algorithm, in each round, clients execute local tasks simultaneously.
- Each local objective function f_i is convex and differentiable.
- Subscripts usually identify clients and superscripts usually refer to round number. See for example the next bullet point.
- During round m , client i computes g_i^m , its **local stochastic gradient**, which estimates its **local gradient** $\nabla f_i(\beta^m)$, using a mini-batch from D_i . We assume that the data points in the mini-batch are drawn uniformly and independently from D_i , ensuring $\mathbb{E} g_i^m = \nabla f_i(\beta^m)$. In the special case where the mini-batch is all of D_i , we have $g_i^m = \nabla f_i(\beta^m)$.
- Clients communicate along the edges of a **communication graph**, an undirected graph $\mathcal{G} = ([K], \mathcal{E})$ where vertices in $[K]$ represent clients and edges in $\mathcal{E} \subseteq \{\{i, j\} : i, j \in [K]\}$ represent communication links. During round m , client i **broadcasts** its gradient g_i^m and/or its model β_i^m to its **neighbors**, members of the set

$$\mathcal{N}_i = \{j : \{i, j\} \in E\} \cup \{i\}.$$

We regard a client as being a neighbor of itself.

- Federated learning methods can be categorized as either **centralized (CFL)** or **decentralized (DFL)**. We note the following differences between CFL and DFL:
 - In CFL all clients will learn the same **global model** $\hat{\beta}$. In DFL each client will learn a distinct **local model** $\hat{\beta}_i$.
 - In CFL there is a distinct client called the **server** which does not possess its own local dataset, but does perform aggregation and model updates. In DFL there is no central server, and clients perform aggregation and model updates themselves.
 - CFL occurs on the **star-shaped graph**, with the server in the middle. DFL occurs on an arbitrary graph.
- During round m , having **fetched** local gradients g_j^m and/or local models β_j^m from neighbors $j \in \mathcal{N}_i$, client i **aggregates** gradients g_j^m and/or models β_j^m , i.e., takes a weighted sum, possibly of the form

$$\sum_{j \in \mathcal{N}_i} w_j^m g_j^m \quad \text{or} \quad \sum_{j \in \mathcal{N}_i} w_j^m \beta_j^m.$$

- During round m , having aggregated local gradients g_j^m and/or local models β_j^m for $j \in \mathcal{N}_i$, client i performs (possibly multiple) **model update(s)**, i.e., assigns to β_i^{m+1} an updated model, based on the current model β_i^m and (usually) the aggregated gradient and/or model. For example,

$$\beta_i^{m+1} \leftarrow \beta_i - w_i g_i^m \quad \text{or} \quad \beta_i^{m+1} \leftarrow \beta_i^m - \sum_{j \in \mathcal{N}_i} w_j^m g_j^m \quad \text{or} \quad \beta_i^{m+1} \leftarrow \sum_{j \in \mathcal{N}_i} w_j^m \beta_j^m.$$

In the above, the first example is (non-federated) gradient descent. The third example should be reminiscent of FedAvg.

Several examples are given in the rest of sections 2 and 3.

2.1 Examples of CFL

Two of the most basic examples of CFL are FederatedAveraging (FedAvg) and FederatedSGD (FedSGD) [MMR16]. In **FedAvg**, clients send local models to the server, which sends back an aggregated global model to the clients. We have the training loop

1. $g_i^m \leftarrow \nabla f_i(\beta^m)$ (clients estimate local gradients)
2. $\beta_i^{m+1/2} \leftarrow \beta^m - \eta^m g_i^m$ (clients do **GD-like update**)
3. $\beta^{m+1} \leftarrow \frac{1}{n} \sum_{i=1}^K n_i \beta_i^{m+1/2}$. (server does **FedAvg update**)

By contrast, in **FedSGD**, clients send gradients to the server, which are aggregated by the central server and applied to the global model, which is sent back to the clients. We have the training loop

1. $g_i^m \leftarrow \nabla f_i(\beta^m)$ (clients estimate local gradients)
2. $\beta^{m+1} \leftarrow \beta^m - \frac{\eta^m}{n} \sum_{i=1}^K n_i g_i^m$. (server does **FedSGD update**)

Here and elsewhere, we adopt the following assumptions and notations:

1. The left squiggly arrow in $g_i^m \leftarrow \nabla f_i(\beta^m)$ denotes that we are assigning to g_i^m an unbiased estimate of $\nabla f_i(\beta^m)$, which is not necessarily the exact value of $\nabla f_i(\beta^m)$.
2. Client-side updates occur synchronously and in parallel.
3. Prior to server-side updates, clients fetch gradients and/or models from their neighbors as necessary.

2.2 Examples of DFL

Many examples of DFL resemble FedAvg, except aggregation occurs over $\mathcal{N}_i$ instead of $[K]$. For example, the training loop of **Decentralized SGD (D-SGD)** [KLB21] is

1. $g_i^m \leftarrow \nabla f_i(\beta_i^m)$ (clients estimate local gradients)
2. $\beta_i^{m+1/2} \leftarrow \beta_i^m - \eta^m g_i^m$ (clients do GD-like update)

$$3. \beta_i^{m+1} \leftarrow \sum_{j \in \mathcal{N}_i} W_{ij} \beta_j^{m+1/2}. \quad (\text{clients do FedAvg-like update})$$

for weight matrix W . The training loop of **Decentralized Parallel SGD (D-PSGD)** [LZZ17] is

$$1. g_i^m \leftarrow \nabla f_i(\beta_i^m) \quad (\text{clients estimate local gradients})$$

$$2. \beta_i^{m+1/2} \leftarrow \sum_{j \in \mathcal{N}_i} W_{ij} \beta_j^m \quad (\text{clients do FedAvg-like update})$$

$$3. \beta_i^{m+1} \leftarrow \beta_i^{m+1/2} - \eta^m g_i^m. \quad (\text{clients do GD-like update})$$

3 Byzantine Failures

Federated learning methods such as FedSGD are susceptible to **Byzantine failures**, in which **Byzantine** clients send arbitrary vectors denoted by $*$ rather than their local stochastic gradient and/or local model. Byzantine clients might do this intentionally to prevent convergence, or they might be well-intentioned but unaware that their local data was poisoned. The set of Byzantine clients is denoted by $\mathcal{B}$ and the set of **honest** clients is denoted by $\mathcal{H}$. The set of Byzantine neighbors of client i is denoted by $\mathcal{B}_i$ and the set of honest neighbors of client i is denoted by $\mathcal{H}_i$. At round m , neighbors of client i will broadcast to client i

$$g_{ij}^m = \begin{cases} g_j^m, & j \in \mathcal{H}_i \\ *, & j \in \mathcal{B}_i, \end{cases}$$

$$\beta_{ij}^m = \begin{cases} \beta_j^m, & j \in \mathcal{H}_i \\ *, & j \in \mathcal{B}_i. \end{cases}$$

In other words, g_j^m is the actual local stochastic gradient of client j , and g_{ij}^m is what client j broadcasts to client i and claims is their local stochastic gradient. This claim is true if $j \in \mathcal{H}_i$, and not necessarily true if $j \in \mathcal{B}_i$. Likewise for β_j^m and β_{ij}^m .

3.1 Byzantine Tolerance

An FL method that defends against Byzantine failures is called **Byzantine-tolerant**. The aggregation mechanisms of Byzantine tolerant FL methods are broadly categorized as being “resilient” or “trust-based” (the categories are not mutually exclusive). In subsubsection 3.1.1 we discuss resilient aggregation and in subsubsection 3.1.2 we discuss trust-based aggregation.

3.1.1 Resilient Aggregation

An aggregation mechanism is **resilient** if it is less sensitive to outlying gradients/models. In CFL, three resilient aggregation mechanisms are median and trimmed mean [YCR18], and Krum [BEG17]. In **median**, gradients are aggregated

$$g = \text{median}(\nabla f_1(\beta), \dots, \nabla f_K(\beta)),$$

where the median is componentwise. In **trimmed mean**, local gradients are aggregated

$$g = \text{trmean}_h(\nabla f_1(\beta), \dots, \nabla f_K(\beta)),$$

where the trimmed mean is applied componentwise, and the h -trimmed mean of scalars $x_1, \dots, x_K$ is defined to be the mean of $x_1, \dots, x_K$ after its largest h and smallest h elements are removed.

Krum is defined as follows. Denote by Γ_i the $K - h - 2$ closest gradients to g_i in Euclidean distance. Define the score $s(i) = \sum_{j \in \Gamma_i} \|g_i - g_j\|_2^2$. We have the aggregation

$$g = \text{Kr}(g_1, \dots, g_K)$$

where Kr selects the g_i with the smallest score.

In DFL, one resilient aggregation mechanism is **ClippedGossip** [HKJ23]:

$$\beta_i^{m+1} = \sum_{j \in \mathcal{N}_i} W_{ij} (\beta_i^m + \text{Clip}(\beta_j^m - \beta_i^m, \tau_i))$$

where for vector x and scalar τ , the **clipping operation** is defined as

$$\text{Clip}(x, \tau) = \min\{1, \tau/\|x\|\} \cdot x, \quad (1)$$

i.e., if $\|x\|$ is bigger than τ , shrink x to have length τ - otherwise, if $\|x\|$ is smaller than τ , leave x alone. We will see in subsubsection 4.2.1 that clipping is a useful tool against magnitude attacks.

3.1.2 Trust

In **trust-based aggregation**, aggregation takes the form of a weighted sum, in which clients assign weights called **trust scores** to their neighbors. In CFL, we will denote the server's trust score for client i at round m as t_i^m ; in DFL, we will denote client i 's trust score for $j \in \mathcal{N}_i$ at round m as t_{ij}^m . In this paragraph is an overview of decentralized trust scores - centralized trust scores work analogously. To compute a trust score t_{ij}^m , client i first computes a **judgment** I_{ij}^m of how suspiciously neighbor j has behaved specifically in round m . This judgment can be continuous (e.g., $I_{ij}^m \in [0, 1]$), or binary (i.e., $I_{ij}^m \in \{0, 1\}$). Euclidean distance [CCL19], cosine similarity [CFL20], k -means [LWW21], and Pearson correlation coefficient [LLX21] have all been used to compute trust in Byzantine tolerant methods. Then, the judgment I_{ij}^m is used to compute the trust score t_{ij}^m . The judgment can be used directly as a trust score, i.e. $t_{ij}^m \leftarrow I_{ij}^m$, or trust scores can depend on the history of judgments, i.e., $t_{ij}^m \leftarrow \text{UpdateTrust}((I_{ij}^\ell)_{\ell=1}^m)$. Finally, the trust score t_{ij}^m is used as a coefficient on the j -th term during client i 's aggregation in the m -th round.

FLTrust [CFL20] is a trust-based CFL method, in which it is assumed that the server has its own local dataset D_0 , with which it can compute local gradients $g_0^m = \nabla f_0(\beta^m)$. Then, the aggregation rule is

$$\frac{1}{\sum_{i=1}^K t_i^m} \sum_{i=1}^K t_i^m \bar{g}_i^m$$

where the trust score is defined by

$$t_i^m = \text{ReLU} \left(\frac{\langle g_0^m, g_i^m \rangle}{\|g_0^m\| \|g_i^m\|} \right)$$

and

$$\bar{g}_i^m = \frac{\|g_0^m\|}{\|g_i^m\|} g_i^m.$$

The term $\frac{\langle g_0^m, g_i^m \rangle}{\|g_0^m\| \|g_i^m\|}$ is the **cosine similarity** of g_0^m and g_i^m , i.e., the cosine of the angle formed by g_0^m and g_i^m . The function $g_i^m \mapsto \bar{g}_i^m$ is the same as the clipping operator if $\|g_i^m\| > \|g_0^m\|$, but differs from the clipping operator otherwise.

Trusted Decentralized FL [GTB22] is a trust-based DFL method. They put $I_{ij}^m \leftarrow I_{ij}^{C(m)} \wedge I_{ij}^{D(m)}$ where $I_{ij}^m, I_{ij}^{C(m)}, I_{ij}^{D(m)} \in \{0, 1\}$. Define the **deviation**

$$\text{dev}_{ij}^m = \sum_{\ell \in \mathcal{N}_i} \frac{\|\beta_{i\ell}^m - \beta_{ij}^m\|^2}{|\mathcal{N}_i|}$$

Then the **clustering-based judgment** is

$$I_{ij}^{C(m)} = \mathbf{1}\{\text{dev}_{ij}^m \leq \text{th}_i \cdot \text{median}\{\text{dev}_{ij}^m\}\},$$

i.e., benchmark dev_{ij}^m against a multiple of the median of all the deviations.

For **difference**

$$d_{ij}^m = \beta_{ij}^m - \beta_i^m, \quad (2)$$

define client i 's **distance-based judgment** of client j

$$I_{ij}^{D(m)} = \mathbf{1}\{\|d_{ij}^m\| \leq \|d_{ij}^{m-1}\|\},$$

which is 1 whenever the distance between β_{ii} and β_{ij} is decreasing between rounds and 0 otherwise.

For $j \in \mathcal{N}_i$, client i puts

$$\tau_{ij}^m = \frac{r_{ij}^m + 1}{r_{ij}^m + s_{ij}^m + 2} \in [0, 1].$$

The quantity τ_{ij} is called the “local trust” client i has for client j , and is based only on the similarity between the models of client i and client j . Client i increments $r_{ij}^{m+1} \leftarrow \rho_1 r_{ij}^m + I_{ij}^{m+1}$ and $s_{ij}^{m+1} \leftarrow \rho_2 s_{ij}^m + 1 - I_{ij}^{m+1}$ where $0 < \rho_1 < \rho_2$ are “forgetting factors”. Client i puts

$$t_{ij}^m = \frac{1}{\sum_{\ell \in \mathcal{N}_i \setminus \{j\}} \tau_{i\ell}^m} \sum_{\ell \in \mathcal{N}_i \setminus \{j\}} \tau_{i\ell}^m t_{\ell j}^{m-1},$$

The quantity t_{ij} is called the “global trust” client i has for client j , and is based on the opinions the other neighbors of client i have for client j . Then, aggregation is essentially that of D-SGD, where the weights are determined by sample size and trust:

$$\beta_i^{m+1} = \frac{1}{\sum_{j \in \mathcal{N}_i} n_j t_{ij}^m} \sum_{j \in \mathcal{N}_i} n_j t_{ij}^m \beta_j^{m+1/2}.$$

4 Main Algorithm

Previously, we have seen some FL methods that use multiple model updates, sequentially. For example, D-SGD involves a GD-like update followed by a FedAvg-like update, and D-PSGD involves a FedAvg-like update followed by a GD-like update. In subsection 4.1, we temporarily ignore the Byzantine setting and discuss model update sequences, including the above two and four others. These correspond to six formulations of the main algorithm. In subsection 4.2, we return to the Byzantine setting, and discuss Byzantine tolerant aggregation mechanisms. In subsection 4.3, we state the main algorithm, informed by the preceding discussions on model update sequences and Byzantine tolerant aggregation.

4.1 Model Updates

In this subsection, we temporarily ignore the Byzantine setting and discuss model update sequences. In subsubsection 4.1.1, we will discuss three basic model updates, which we will denote by G, A, and S. In subsubsection 4.1.2, we will discuss carrying out multiple model updates simultaneously, and in subsubsection 4.1.3 we will discuss carrying out multiple model updates sequentially.

4.1.1 G, A, and S

We have seen previously three basic model updates. These were the non-federated **GD-like update (G)**

$$\beta_i^{m+1} \leftarrow \beta_i^m - \eta^m g_i^m, \quad (\text{G})$$

the **FedAvg-like update (A)**, which involves aggregating models (the d_{ij}^m is a difference of models as in equation 2)

$$\beta_i^{m+1} \leftarrow \beta_i^m + \frac{\eta^m}{\sum_{j \in \mathcal{N}_i} t_{ij}^m} \sum_{j \in \mathcal{N}_i} t_{ij}^m d_{ij}^m, \quad (\text{A})$$

and the **FedSGD-like update (S)**, which involves aggregating gradients

$$\beta_i^{m+1} \leftarrow \beta_i^m - \frac{\eta^m}{\sum_{j \in \mathcal{N}_i} t_{ij}^m} \sum_{j \in \mathcal{N}_i} t_{ij}^m g_{ij}^m. \quad (\text{S})$$

4.1.2 Simultaneous Model Updates

We can carry out multiple of the three basic model updates simultaneously. Since S already involves the term g_{ii} , it would be redundant to carry out G and S simultaneously. This leaves us with **AG update (AG)** and **AS update (AS)**

$$\beta_i^{m+1} \leftarrow \beta_i^m - \frac{\eta^m}{|\mathcal{N}_i| + \sum_{j \in \mathcal{N}_i} t_{ij}^m} \left(|\mathcal{N}_i| g_i^m - \sum_{j \in \mathcal{N}_i} t_{ij}^m d_{ij}^m \right), \quad (\text{AG})$$

$$\beta_i^{m+1} \leftarrow \beta_i^m - \frac{\eta^m}{2 \sum_{j \in \mathcal{N}_i} t_{ij}^m} \sum_{j \in \mathcal{N}_i} t_{ij}^m (g_{ij}^m - d_{ij}^m). \quad (\text{AS})$$

We now have five model updates: the three basic model updates G, A, S, and the two simultaneous model updates AG, and AS.

4.1.3 Model Update Sequences

We can carry out multiple of the five model updates sequentially. To disambiguate model update sequences and model updates, we will surround model update sequences by quotation marks and separate individual model updates by commas. For example, we distinguish between the model update sequence “AG”, “A,G”, and “G,A”. The notation “AG” denotes the model update sequence consisting only of the model update AG. The notation “A,G” denotes the model update sequence consisting of A followed by G:

$$\begin{cases} \beta_i^{m+1/2} & \leftarrow \beta_i^m + \frac{\eta^m}{\sum_{j \in \mathcal{N}_i} t_{ij}^m} \sum_{j \in \mathcal{N}_i} t_{ij}^m d_{ij}^m \\ \beta_i^{m+1} & \leftarrow \beta_i^{m+1/2} - \eta^{m+1/2} g_i^{m+1/2}. \end{cases} \quad (\text{“A,G”})$$

The notation “G,A” denotes the model update sequence consisting of G followed by A:

$$\begin{cases} \beta_i^{m+1/2} & \leftarrow \beta_i^m - \eta^m g_i^m \\ \beta_i^{m+1} & \leftarrow \beta_i^m + \frac{\eta^{m+1/2}}{\sum_{j \in \mathcal{N}_i} t_{ij}^{m+1/2}} \sum_{j \in \mathcal{N}_i} t_{ij}^{m+1/2} d_{ij}^{m+1/2}. \end{cases} \quad (\text{“G,A”})$$

The model update sequences “AG”, “A,G”, and “G,A” are distinct because the vectors they aggregate are distinct. Note that “A,G” includes a term $g_i^{m+1/2}$, “G,A” includes terms $d_{ij}^{m+1/2}$, and “AG” includes neither. We have

$$\begin{aligned} g_i^m &\approx \nabla f_i(\beta_i^m) \\ g_i^{m+1/2} &\approx \nabla f_i(\beta_i^{m+1/2}) \\ d_{ij}^m &= \beta_{ij}^m - \beta_i^m \\ d_{ij}^{m+1/2} &= \beta_{ij}^{m+1/2} - \beta_i^{m+1/2}. \end{aligned}$$

We have four model update sequences consisting of just one model update (“G”, “S”, “AG”, and “AS”). We note that the model update sequence “A” is invalid. This is because any valid model update sequence must involve a computation of a gradient at some point, so any valid model update sequence must include the letter “G” or the letter “S”. We have also seen two model update sequences consisting of two model updates sequentially (“A,G” and “G,A”). It turns out that these six model update sequences are the only valid model update sequences subject to the following list of common sense constraints:

1. **Gradient method.** Belonging to a gradient method, the model update sequence must contain at least one of the letters G or S.
2. **Information recency.** A fetch should immediately precede whichever is present among A, S, AG, or AS. This is mainly relevant for the model update sequence “G,A”, in which the fetch occurs after the G but before the A. By contrast, in the model update sequence “A,G”, the fetch occurs before the A (and also before the G).
3. **Communication cost.** To curtail communication cost, clients should fetch exactly once per round. For example, this rules out “A,S”, as a fetch would have to precede the A and the S.
4. **Computational cost.** To curtail computational cost, clients should compute a gradient exactly once per round.

If we further stipulate that the model update sequence belongs to a federated method, then the model update sequence “G” is also ruled out. However, we continue to consider “G” so we can compare the federated methods to a non-federated baseline during numerical experiments.

In table 1, we summarize existing aggregation mechanisms.

Method	Sequence
GD	“G”
FedAvg, D-SGD, ClippedGossip, Trusted Decentralized FL	“G,A”
FedSGD, Median, Trimmed mean, Krum, FLTrust	“S”
D-PGSD	“AG”

Table 1: Summary of existing aggregation mechanisms

4.2 Byzantine Tolerance

In this subsection, we return to the Byzantine setting. We recall that Byzantine tolerant aggregation mechanisms are broadly categorized as being resilient and/or trust-based. We give two naive training loops as motivating examples. In subsubsection 4.2.1, we give a naive training loop which only employs trust scores, and a corresponding Byzantine failure which prevents convergence. In subsubsection 4.2.2, we give a naive training loop which only employs clipping (a kind of resilient aggregation), and a corresponding Byzantine failure which prevents convergence. In light of these examples, the main algorithm employs both trust scores and clipping.

4.2.1 Trust Scores Are Insufficient

Consider the naive decentralized training loop

1. $g_{ij}^m \leftarrow \begin{cases} g_j^m, & j \in \mathcal{H}_i \\ *, & j \in \mathcal{B}_i \end{cases}$ (client estimates local gradient)
2. $G_i^m \leftarrow \frac{1}{\sum_{j \in \mathcal{N}_i} t_{ij}^m} \sum_{j \in \mathcal{N}_i} t_{ij}^m g_{ij}^m$ (clients aggregate neighbors' suggestions)
3. $\beta_i^{m+1} \leftarrow \beta_i^m - \eta^m G_i^m$ (clients apply aggregated suggestion)

where for $j \in \mathcal{N}_i$, the trust score $t_{ij}^m \in (0, 1)$ is chosen by client i . A **magnitude attack** is a crude strategy in which a single Byzantine client sends g_{ij}^m with very large magnitude. Specifically, if Byzantine client b wants $G_i^m = h$, it can send the vector

$$g_{ib}^m = \frac{1}{t_{ib}^m} \left(\left(\sum_{j \in \mathcal{N}_i} t_{ij}^m \right) h - \sum_{j \in \mathcal{N}_i \setminus \{b\}} t_{ij}^m g_{ij}^m \right).$$

Note that it is insufficient to shrink t_{ib}^m down to a small positive number, as client b can send counteractingly large vectors. I.e., trust scores alone cannot defend against a magnitude attack.

4.2.2 Clipping Is Insufficient

To defend against a magnitude attack, this next naive training loop employs the clipping operator (as in equation 1):

1. $g_{ij}^m \leftarrow \begin{cases} g_j^m, & j \in \mathcal{H}_i \\ *, & j \in \mathcal{B}_i \end{cases}$ (clients estimate local gradients)
2. $\tilde{g}_{ij}^m \leftarrow \text{Clip}(g_{ij}^m, \|g_i^m\|)$ (clients clip neighbors' suggestions)
3. $G_i^m \leftarrow \frac{1}{\sum_{j \in \mathcal{N}_i} t_{ij}^m} \sum_{j \in \mathcal{N}_i} t_{ij}^m \tilde{g}_{ij}^m$ (clients aggregate neighbors' clipped suggestions)
4. $\beta_i^{m+1} \leftarrow \beta_i^m - \eta^m G_i^m$ (clients apply aggregated suggestion)

In the new naive training loop, the magnitude of each contribution $\tilde{g}_{ij}^m$ is bounded above by $\|g_i^m\|$. However, a collection of coordinated Byzantine clients can still prevent convergence. Define $G_h^m = \sum_{j \in \mathcal{H}_i} t_{ij}^m \tilde{g}_{ij}^m$, i.e., the aggregation of honest clipped local stochastic gradients, and suppose G_h^m

points in a descent direction. Suppose Byzantine clients $j \in \mathcal{B}_i$ each send to client i the vector $g_{ij}^m = -G_h^m$. Then,

$$G_i^m = \frac{1}{\sum_{j \in \mathcal{N}_i} t_{ij}^m} \left(G_h^m + \sum_{j \in \mathcal{B}_i} t_{ij}^m \text{Clip}(-G_h^m, \|g_i^m\|) \right).$$

Assuming $\|G_h^m\| > \|g_i^m\|$,

$$\begin{aligned} G_i^m &= \frac{1}{\sum_{j \in \mathcal{N}_i} t_{ij}^m} \left(G_h^m + \left(\sum_{j \in \mathcal{B}_i} t_{ij}^m \right) \frac{\|g_i^m\|}{\|G_h^m\|} (-G_h^m) \right) \\ &= \frac{1}{\sum_{j \in \mathcal{N}_i} t_{ij}^m} \left(1 - \frac{\|g_i^m\|}{\|G_h^m\|} \left(\sum_{j \in \mathcal{B}_i} t_{ij}^m \right) \right) G_h^m. \end{aligned}$$

The vector g_i^m will point in the opposite direction of G_h^m whenever

$$\begin{aligned} 1 - \frac{\|g_i^m\|}{\|G_h^m\|} \left(\sum_{j \in \mathcal{B}_i} t_{ij}^m \right) &< 0 \\ \iff \left(\sum_{j \in \mathcal{B}_i} t_{ij}^m \right) &> \frac{\|G_h^m\|}{\|g_i^m\|} \end{aligned}$$

Note that

$$\begin{aligned} \|G_h^m\| &= \left\| \sum_{j \in \mathcal{H}_i} t_{ij}^m \tilde{g}_{ij}^m \right\| \\ &\leq \sum_{j \in \mathcal{H}_i} t_{ij}^m \|\tilde{g}_{ij}^m\| \\ &\leq \left(\sum_{j \in \mathcal{H}_i} t_{ij}^m \right) \|g_i^m\|. \end{aligned}$$

Hence,

$$\begin{aligned} \left(\sum_{j \in \mathcal{B}_i} t_{ij}^m \right) &> \left(\sum_{j \in \mathcal{H}_i} t_{ij}^m \right) \\ \implies \left(\sum_{j \in \mathcal{B}_i} t_{ij}^m \right) &> \frac{\|G_h^m\|}{\|g_i^m\|}, \end{aligned}$$

i.e., a sufficient condition for G_i^m to point in the opposite direction of G_h^m is that the sum of Byzantine trust scores is greater than the sum of honest trust scores.

Suppose additionally that client i does not employ trust scores, instead putting $t_{ij}^m = 1$, $j \in \mathcal{N}_i$. In this case, $\sum_{j \in \mathcal{B}_i} t_{ij}^m = |\mathcal{B}_i|$ and $\sum_{j \in \mathcal{H}_i} t_{ij}^m = |\mathcal{H}_i|$. The sufficient condition for G_i^m to point in the opposite direction of G_h^m becomes that $|\mathcal{B}_i| > |\mathcal{H}_i|$. I.e., without trust scores, a majority of Byzantine neighbors can cause the aggregated gradient to point opposite to the aggregated honest gradient, even with clipping.

4.3 Main Algorithm

In light of subsections 4.1 and 4.2, we give the basic anatomy of a round of the main algorithm below. During round m , client $i \in \mathcal{H}$ does (for $j \in \mathcal{N}_i$):

1. **Estimate gradient.** Compute local stochastic gradient $g_i^m \approx \nabla f_i(\beta_i^m)$.
2. **Fetch messages from your neighbors.** Depending on the model update sequence, fetch message(s) g_{ij}^m and/or β_{ij}^m . The term **message** refers collectively to g_{ij}^m and β_{ij}^m .
3. **Judge your neighbors.** Depending on whether g_{ij}^m , β_{ij}^m , or both was fetched, perform a judgment taking g_{ij}^m as an argument, a judgment taking β_{ij}^m as an argument, or both, and obtain $I_{ij}^m \in \{0, 1\}$, indicating whether neighbor j acted nonsuspiciously during round m .
4. **Compute trust scores.** Calculate trust scores using the formula $t_{ij}^m = \frac{1}{m} \sum_{k=1}^m I_{ij}^k$.
5. **Clip the suggestions of your neighbors.** Clip the suggestion(s) g_{ij}^m and/or $d_{ij}^m = \beta_{ij}^m - \beta_i^m$ by $\|g_i^m\|$, and obtain **clipped suggestion(s)** $\tilde{g}_{ij}^m$ and/or $\tilde{d}_{ij}^m$. The term **suggestion** refers collectively to g_{ij}^m and d_{ij}^m .
6. **Follow the model update sequence.** Carry out model updates according to the model update sequence, where the vectors are clipped suggestions, and the coefficients are the trust scores t_{ij}^m , and obtain β_i^{m+1} .

4.3.1 Judgment

The way in which clients judge their neighbors in step 3 depends on whether the message(s) they have fetched are (potentially falsified) gradients or models. We conceptualize gradients as arrows that should point toward the optimal model, and local models as points that should move toward the optimal model. Hence, if client i has fetched the gradient of client j , they compute the **angle-based judgment**

$$I_{ij}^{\nabla(m)} = \mathbf{1} \{ \langle g_i^m, g_{ij}^m \rangle > 0 \},$$

which is 1 whenever g_i^m and g_{ij}^m form an acute angle and 0 otherwise. On the other hand, if client i has fetched the local model of client j , they compute the distance-based judgment

$$I_{ij}^{D(m)} = \mathbf{1} \{ \|d_{ij}^m\| \leq \|d_{ij}^{m-1}\| \},$$

which is 1 whenever the distance between β_i and β_{ij} is decreasing between rounds and 0 otherwise.

The proof of Theorem 1 is agnostic to the particular manner by which client i computes judgments I_{ij} , caring only that for $j \in \mathcal{H}_i$, $I_{ij} = 1$ the majority of the time, and for $j \in \mathcal{B}_i$, $I_{ij} = 0$ a majority of the time.

4.3.2 Pseudocode

Algorithms 1, 2, and 3 are helper functions for computing trust.

Algorithm 1 UpdateAngleBasedTrust

Require: $g_i^m, g_{ij}^m, I_{ij}^{\nabla(1)}, \dots, I_{ij}^{\nabla(m-1)}$
 $I_{ij}^{\nabla(m)} \leftarrow \mathbf{1}\{\langle g_i^m, g_{ij}^m \rangle > 0\}$
return $t_{ij}^m \leftarrow \frac{1}{m} \sum_{k=1}^m I_{ij}^{\nabla(k)}$

Algorithm 2 UpdateDistanceBasedTrust

Require: $d_{ij}^m, d_{ij}^{m-1}, I_{ij}^{D(1)}, \dots, I_{ij}^{D(m-1)}$
 $I_{ij}^{D(m)} \leftarrow \mathbf{1}\{\|d_{ij}^m\| \leq \|d_{ij}^{m-1}\|\}$
return $t_{ij}^m \leftarrow \frac{1}{m} \sum_{k=1}^m I_{ij}^{D(k)}$

Algorithm 3 UpdateBothTrust

Require: $g_i^m, g_{ij}^m, I_{ij}^{\nabla(1)}, \dots, I_{ij}^{\nabla(m-1)}, d_{ij}^m, d_{ij}^{m-1}, I_{ij}^{D(1)}, \dots, I_{ij}^{D(m-1)}$
 $I_{ij}^{\nabla(m)} \leftarrow \mathbf{1}\{\langle g_i^m, g_{ij}^m \rangle > 0\}$
 $I_{ij}^{D(m)} \leftarrow \mathbf{1}\{\|d_{ij}^m\| \leq \|d_{ij}^{m-1}\|\}$
return $t_{ij}^m \leftarrow \frac{1}{2m} \sum_{k=1}^m (I_{ij}^{\nabla(k)} + I_{ij}^{D(k)})$

Algorithm 4 gives the versions of the main algorithm without a FedSGD-like update (S).

Algorithm 4 Main algorithm with no FedSGD-like update

Require: GLOBALLY: maximum round number M ; client index set $[K]$; aggregation mechanism “seq”, initial model β^0 , learning rate $(\eta^m)_{m \in \mathbb{N}}$. **LOCALLY** (i.e., each client i holds privately): objective $f_i(\beta)$, initial model $\beta_i^0 = \beta^0$, neighborhood set $\mathcal{N}_i$

for $m \in \{0, \dots, M\}$ **do** {or until stopping condition}

for $i \in [K]$ **do** {in parallel}

if seq == “G” **then**

$g_i^m \leftarrow \nabla f_i(\beta_i^m)$

$\beta_i^{m+1} \leftarrow \beta_i^m - \eta^m g_i^m$

else if seq == “AG” **then**

$g_i^m \leftarrow \nabla f_i(\beta_i^m)$

 Fetch β_{ij}^m for $j \in \mathcal{N}_i$

$d_{ij}^m \leftarrow \beta_{ij}^m - \beta_i^m$ and $\tilde{d}_{ij}^m \leftarrow \text{Clip}(d_{ij}^m, \|g_i^m\|)$ for $j \in \mathcal{N}_i$

$t_{ij}^m \leftarrow \text{UpdateDistanceBasedTrust}(d_{ij}^m, d_{ij}^{m-1}, I_{ij}^{D(1)}, \dots, I_{ij}^{D(m-1)})$ for $j \in \mathcal{N}_i$

$\beta_i^{m+1} \leftarrow \beta_i^m - \frac{\eta^m}{|\mathcal{N}_i| + \sum_{j \in \mathcal{N}_i} t_{ij}^m} \left(|\mathcal{N}_i| g_i^m - \sum_{j \in \mathcal{N}_i} t_{ij}^m \tilde{d}_{ij}^m \right)$

else if seq == “A,G” **then**

 Fetch β_{ij}^m for $j \in \mathcal{N}_i$

$d_{ij}^m \leftarrow \beta_{ij}^m - \beta_i^m$ and $\tilde{d}_{ij}^m \leftarrow \text{Clip}(d_{ij}^m, \|g_i^m\|)$ for $j \in \mathcal{N}_i$

$t_{ij}^m \leftarrow \text{UpdateDistanceBasedTrust}(d_{ij}^m, d_{ij}^{m-1}, I_{ij}^{D(1)}, \dots, I_{ij}^{D(m-1)})$ for $j \in \mathcal{N}_i$

$\beta_i^{m+1/2} \leftarrow \beta_i^m + \frac{\eta^m}{\sum_{j \in \mathcal{N}_i} t_{ij}^m} \sum_{j \in \mathcal{N}_i} t_{ij}^m \tilde{d}_{ij}^m$

$g_i^{m+1/2} \leftarrow \nabla f_i(\beta_i^{m+1/2})$

$\beta_i^{m+1} \leftarrow \beta_i^{m+1/2} - \eta^m g_i^{m+1/2}$

else if seq == “G,A” **then**

$g_i^m \leftarrow \nabla f_i(\beta_i^m)$

$\beta_i^{m+1/2} \leftarrow \beta_i^m - \eta^m g_i^m$

 Fetch $\beta_{ij}^{m+1/2}$ for $j \in \mathcal{N}_i$

$d_{ij}^{m+1/2} \leftarrow \beta_{ij}^{m+1/2} - \beta_i^{m+1/2}$ and $\tilde{d}_{ij}^{m+1/2} \leftarrow \text{Clip}(d_{ij}^{m+1/2}, \|g_i^m\|)$ for $j \in \mathcal{N}_i$

$t_{ij}^{m+1/2} \leftarrow \text{UpdateDistanceBasedTrust}(d_{ij}^{m+1/2}, d_{ij}^{m-1/2}, I_{ij}^{D(1)}, \dots, I_{ij}^{D(m-1)})$ for $j \in \mathcal{N}_i$

$\beta_i^{m+1} \leftarrow \beta_i^{m+1/2} + \frac{\eta^m}{\sum_{j \in \mathcal{N}_i} t_{ij}^{m+1/2}} \sum_{j \in \mathcal{N}_i} t_{ij}^{m+1/2} \tilde{d}_{ij}^{m+1/2}$

return $\beta_1^M, \dots, \beta_K^M$

Algorithm 5 gives the versions of the main algorithm with a FedSGD-like update (S).

Algorithm 5 Main algorithm with FedSGD-like update

Require: GLOBALLY: maximum round number M ; client index set $[K]$; aggregation mechanism “seq”, initial model β^0 , learning rate $(\eta^m)_{m \in \mathbb{N}}$. LOCALLY (i.e., each client i holds privately): objective $f_i(\beta)$, initial model $\beta_i^0 = \beta^0$, neighborhood set $\mathcal{N}_i$

for $m \in \{0, \dots, M\}$ **do** {or until stopping condition}

for $i \in [K]$ **do** {in parallel}

if seq == “S” **then**

$g_i^m \leftarrow \nabla f_i(\beta_i^m)$

 Fetch g_{ij}^m for $j \in \mathcal{N}_i$

$t_{ij}^m \leftarrow \text{UpdateAngleBasedTrust}(g_i^m, g_{ij}^m, I_{ij}^{\nabla(1)}, \dots, I_{ij}^{\nabla(m-1)})$ for $j \in \mathcal{N}_i$

$\tilde{g}_{ij}^m \leftarrow \text{Clip}(g_{ij}^m, \|g_i^m\|)$ for $j \in \mathcal{N}_i$

$\beta_i^{m+1} \leftarrow \beta_i^m - \frac{\eta^m}{\sum_{j \in \mathcal{N}_i} t_{ij}^m} \sum_{j \in \mathcal{N}_i} t_{ij}^m \tilde{g}_{ij}^m$

else if seq == “AS” **then**

$g_i^m \leftarrow \nabla f_i(\beta_i^m)$

 Fetch g_{ij}^m, β_{ij}^m for $j \in \mathcal{N}_i$

$d_{ij}^m \leftarrow \beta_{ij}^m - \beta_i^m$ and $\tilde{d}_{ij}^m \leftarrow \text{Clip}(d_{ij}^m, \|g_i^m\|)$ for $j \in \mathcal{N}_i$

$t_{ij}^m \leftarrow \text{UpdateBothTrust}(g_i^m, g_{ij}^m, I_{ij}^{\nabla(1)}, \dots, I_{ij}^{\nabla(m-1)}, d_{ij}^m, d_{ij}^{m-1}, I_{ij}^{D(1)}, \dots, I_{ij}^{D(m-1)})$ for $j \in \mathcal{N}_i$

$\tilde{g}_{ij}^m \leftarrow \text{Clip}(g_{ij}^m, \|g_i^m\|)$ for $j \in \mathcal{N}_i$

$\beta_i^{m+1} \leftarrow \beta_i^m - \frac{\eta^m}{\sum_{j \in \mathcal{N}_i} t_{ij}^m} \sum_{j \in \mathcal{N}_i} t_{ij}^m (\tilde{g}_{ij}^m - \tilde{d}_{ij}^m)$

return $\beta_1^M, \dots, \beta_K^M$

4.3.3 Novel Contributions

To the best of our knowledge the novel contributions of the main algorithm are the following:

- Consideration of the model update sequences “A,G” and “AS”.
- Consideration of the model update sequence “S” in a DFL setting.
- Using both clipping and trust scores which depend on the history of judgments, in a DFL setting.

In section 5 we will see a Byzantine tolerant DFL method called Square Loss Byzantine-Robust Network (SLBRN) [ZW24] that specifically solves a LASSO objective function and has model update sequence “A,S”. This model update sequence was not one we considered because it would requires two fetches in a round: one preceding the A and one preceding the S. Another difference between the main algorithm and SLBRN is that SLBRN does not use trust scores.

In table 2, we compare and contrast the main algorithm of this exam to existing Byzantine tolerant methods.

Method	Topology	Resilient Agg.	Judgment	Trust	Agg. Seq.	LASSO
ClippedGossip	DFL	clipping	no	no	"G,A"	no
FLTrust	CFL	normalize ∇ 's	cosine	depends on most recent judgment	"S"	no
Trusted Decentralized FL	DFL	no	clustering, distance	depends on history of judgments	"G,A"	no
SLBRN	DFL	median/trimmed mean/Krum	no	no	"A,S"	yes
Main Algorithm of Exam	DFL	clipping	angle, distance	depends on history of judgments	multiple	yes

Table 2: Summary of Byzantine tolerant methods

4.4 Complexity Analysis

The complexity of the main algorithm depends on the loss function so we will for sake of example assume squared loss

$$f_i(\beta) = \frac{1}{2n_i} \|Y_i - X_i\beta\|^2,$$

where $X_i \in \mathbb{R}^{n_i \times p}$, $Y_i \in \mathbb{R}^{n_i}$, and $\beta \in \mathbb{R}^p$. We compute the complexity of the basic steps of a round of the main algorithm:

1. **Estimate gradient.** The gradient is

$$\nabla f_i(\beta) = \frac{1}{n_i} X_i^\top (X_i\beta - Y_i)$$

with complexity $O(n_i p)$. Putting $n_{\max} := \max_i n_i$, this is bounded above by $O(n_{\max} p)$.

2. **Judge your neighbors.** The operations

$$\begin{aligned} I_{ij}^\nabla &= \mathbf{1} \{ \langle g_{ii}, g_{ij} \rangle > 0 \} \\ I_{ij}^D &= \mathbf{1} \{ \|d_{ij}^m\| \leq \|d_{ij}^{m-1}\| \} \end{aligned}$$

both have complexity $O(p)$. Clients, in parallel, compute judgments once per neighbor per round, so putting $\mathcal{N}_{\max} := \max_i |\mathcal{N}_i|$, we have complexity $O(\mathcal{N}_{\max} p)$.

3. **Compute trust scores.** Each trust score has complexity $O(1)$, and clients, in parallel, compute trust scores once per neighbor per round, so we have complexity $O(\mathcal{N}_{\max})$.
4. **Clip the suggestions of your neighbors.** The clipping operator is $O(p)$, and clients, in parallel, clip once per neighbor per round, so we have complexity $O(\mathcal{N}_{\max} p)$.
5. **Follow the model update sequence.** The model updates A, S, AG, and AS, all have complexity $O(\mathcal{N}_{\max} p)$, and the model update sequence consists of up to two of these model updates. Clients, in parallel, follow the model update sequence once per round, so we have complexity $O(\mathcal{N}_{\max} p)$.

Then, given the number of rounds M , the complexity of the main algorithm is $O(M(n_{\max} + \mathcal{N}_{\max})p)$.

5 Proximal Gradient Methods

The background on proximal gradient methods given in this section comes from [PB14].

Consider minimizing the objective function

$$f(\beta) = F(\beta) + R(\beta), \tag{3}$$

where $F: \mathbb{R}^p \rightarrow \mathbb{R}$ and $R: \mathbb{R}^p \rightarrow \mathbb{R}$ are closed proper convex and f is L -smooth. A point β^* minimizes f if and only if, for any $\eta > 0$,

$$\beta^* = \text{prox}_{\eta R}(\beta^* - \eta \nabla F(\beta^*)),$$

where the **proximal operator** is

$$\text{prox}_{\eta R}(\beta) = \arg \min_x \left(\eta R(\beta) + \frac{\|x - \beta\|^2}{2} \right).$$

The **proximal gradient method** is

$$\beta^{m+1} \leftarrow \text{prox}_{\eta^m R}(\beta^m - \eta^m \nabla F(\beta^m)),$$

where $\eta^m > 0$ is a step size. This method converges to β^* with rate $O(1/m)$ for fixed step size $\eta^m = \eta \in (0, 1/L]$.

The main algorithm is a gradient method, requiring that f_i is L -smooth for $i \in [K]$. To handle local objective functions that can be split as in Equation 3, the main algorithm is modified in two ways:

1. Replace ∇f_i with the gradient of the **differentiable part**, ∇F_i .
2. Append a proximal step to each round.

I.e., during round m , client $i \in \mathcal{H}$ does (for $j \in \mathcal{N}_i$):

1. **Estimate gradient.** Compute local stochastic gradient $g_i^m \approx \nabla F_i(\beta_i^m)$.
2. **Fetch messages from your neighbors.** Depending on the model update sequence, fetch message(s) g_{ij}^m and/or β_{ij}^m .
3. **Judge your neighbors.** Depending on whether g_{ij}^m , β_{ij}^m , or both was fetched, perform a judgment taking g_{ij}^m as an argument, a judgment taking β_{ij}^m as an argument, or both, and obtain $I_{ij}^m \in \{0, 1\}$, indicating whether neighbor j acted nonsuspiciously during round m .
4. **Compute trust scores.** Calculate trust scores using the formula $t_{ij}^m = \frac{1}{m} \sum_{k=1}^m I_{ij}^k$.
5. **Clip the suggestions of your neighbors.** Clip the suggestion(s) g_{ij}^m and/or $d_{ij}^m = \beta_{ij}^m - \beta_i^m$ by $\|g_i^m\|$, and obtain **clipped suggestion(s)** $\tilde{g}_{ij}^m$ and/or $\tilde{d}_{ij}^m$.
6. **Follow the model update sequence.** Carry out model updates according to the model update sequence, where the vectors are clipped suggestions, and the coefficients are the trust scores t_{ij}^m , and obtain β^{m+1} .
7. **Apply proximal operator.** Put $\beta_i^{m+1} \leftarrow \text{prox}_{\eta^m R}(\beta_i^{m+1})$.

5.1 LASSO

Solving the LASSO problem is the illustrative example of proximal gradient methods and the subject of our numerical experiments in section 7. Assume that each client $i \in [K]$ possesses $D_i = (X_i, Y_i)$, where

$$X_i = \begin{bmatrix} x_{i1} \\ \vdots \\ x_{in_i} \end{bmatrix} \in \mathbb{R}^{n_i \times p}$$

$$Y_i = \begin{bmatrix} y_{i1} \\ \vdots \\ y_{in_i} \end{bmatrix} \in \mathbb{R}^{n_i},$$

and their local objective function is a LASSO objective function:

$$f_i(\beta) = F_i(\beta) + \lambda \|\beta\|_1 = \frac{1}{2n_i} \|Y_i - X_i \beta\|^2 + \lambda \|\beta\|_1.$$

The proximal operator of $\lambda \|\cdot\|_1$ is the **soft-thresholding operator**, defined componentwise by

$$[S_\lambda(x)]_i = \begin{cases} x_i - \lambda, & x_i > \lambda \\ 0, & |x_i| \leq \lambda \\ x_i + \lambda, & x_i < -\lambda. \end{cases}$$

We will show in section 6 that the main algorithm with a soft-thresholding step converges to the minimizer of the global objective function

$$f(\beta) = \frac{1}{K} \sum_{i=1}^K \frac{1}{2n_i} \|Y_i - X_i \beta\|^2 + \lambda \|\beta\|_1.$$

One existing Byzantine-tolerant DFL method solving the LASSO problem is **Square Loss Byzantine-Robust Network (SLBRN)** [ZW24]. The training loop for SLBRN is

1. $\beta_i^m \leftarrow \mathcal{A} \left(\{\beta_{ij}^{m-1/2}\}_{j \in \mathcal{N}_i} \right)$ (clients do FedAvg-like update)
2. $g_i^m \leftarrow G_i^{m-1} + \nabla F_i(\beta_i^m) - \nabla F_i(\beta_i^{m-1})$ (clients compute gradients)
3. $G_i^m \leftarrow \mathcal{A} \left(\{g_{ij}^m\}_{j \in \mathcal{N}_i} \right)$
4. $\beta_i^{m+1/2} \leftarrow \beta_i^m - \frac{1}{\gamma} G_i^m$ (clients do FedSGD-like update)
5. $\beta_i^{m+1/2} \leftarrow S_\lambda(\beta_i^{m+1/2})$ (clients do soft-thresholding update)

where $\mathcal{A}$ is an aggregation mechanism chosen from median, trimmed mean, and Krum (taken over $\mathcal{N}_i$), and γ is a penalty parameter. In step 2, SLBRN employs “gradient tracking” to estimate the global gradient. The model update sequence of SLBRN is “A,S” because it involves a FedAvg-like update followed by a FedSGD-like update. In SLBRN, clients have to fetch twice per round: fetch models before the FedAvg-like update, and gradients before the FedSGD-like update.

6 Convergence Analysis

In Appendix A, we prove Theorem 1, which states that with constant step size $0 < \eta < \min \left\{ \frac{\mu}{4L^2}, \frac{2}{\mu} \right\}$, we have

$$\mathbb{E} \|\beta_i^m - \beta^*\|^2 \leq \left(1 - \frac{\eta\mu}{2}\right)^m \|\beta_i^0 - \beta^*\|^2 + \frac{10}{\mu^2} (2L^2 \|\beta^0 - \beta^*\|^2 + \sigma^2),$$

assuming F is μ -strongly convex and L -smooth, and local stochastic gradients have bounded variance

$$\mathbb{E} \|g_i^m\|^2 \leq \|\nabla F(\beta_i^m)\|^2 + \sigma^2.$$

7 Numerical Experiments

We evaluate the main algorithm with four numerical experiments. The first investigates the effect of the number of adversaries. The second investigates the Byzantine tradeoff between impacting your neighbors' models and appearing trustworthy to your neighbors. The third investigates the effect of the connectedness of the communication graph. The fourth investigates the effect of different kinds of Byzantine failures. Find code at <https://github.com/gilbert-neuner/decentralized-federated-learning>.

7.1 Setup

7.1.1 Data Generation

Across all experiments, each client has the same sample size $n_1 = \dots = n_K = 50$ (the number of clients K may vary), and dimension $p = 100$. For $i \in [K]$, the outcome $Y_i \in \mathbb{R}^{50}$ is generated from the linear regression model

$$Y_i = X_i \beta^* + \epsilon_i$$

for $X_i \in \mathbb{R}^{100 \times 50}$, $\beta^* \in \mathbb{R}^{50}$, and $\epsilon_i \in \mathbb{R}^{100}$. The true model $\beta^* \in \mathbb{R}^{100}$ is a vector of 95 zeros and 5 ones, the positions of the ones drawn uniformly without replacement from $[100]$. I.e., the sparsity of β^* is 0.05. For $i \in [K]$, each row of $X_i \in \mathbb{R}^{50 \times 100}$ is drawn i.i.d. from the multivariate normal distribution $N_{100}(0_{100}, I_{100})$. For $i \in [K]$, each component of the error $\epsilon_i \in \mathbb{R}^{100}$ is drawn i.i.d. from the univariate normal distribution $N_1(0, \|\beta^*\|^2)$. The variance is chosen to be $\|\beta^*\|^2$ so that the signal to noise ratio equals 1. Since each Y_i is generated from a linear regression model and β^* is sparse, local objective functions are LASSO objective functions

$$f_i(\beta) = \frac{1}{100} \|Y_i - X_i \beta\|^2 + \lambda \|\beta\|_1.$$

The global objective function is

$$f(\beta) = \frac{1}{10} \sum_{i=1}^{10} \frac{1}{100} \|Y_i - X_i \beta\|^2 + \lambda \|\beta\|_1 = \frac{1}{1000} \|Y - X \beta\|^2 + \lambda \|\beta\|_1,$$

where the **global dataset** is defined by

$$X = \begin{bmatrix} X_1 \\ \vdots \\ X_{10} \end{bmatrix} \in \mathbb{R}^{500 \times 100}$$

$$Y = \begin{bmatrix} Y_1 \\ \vdots \\ Y_{10} \end{bmatrix} \in \mathbb{R}^{500}.$$

The global objective function is itself a LASSO objective function, hence optimizable with proximal gradient descent.

7.1.2 Performance Metrics

In the LASSO problem, two targets are the support of β^* and β^* itself. To evaluate recovery of the support of β^* , we use **precision**, **recall**, and F_1 :

$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad F_1 = \frac{2\text{TP}}{2\text{TP} + \text{FP} + \text{FN}}.$$

Precision penalizes false positives, recall penalizes false negatives, and F_1 , being their harmonic mean, balances both considerations.

In a replicate, TP's, FP's, and FN's are tallied across all honest clients. Entries of β^* and $\hat{\beta}$ greater than zero correspond to actual positives and predicted positives, respectively, and entries of β^* and $\hat{\beta}$ less than or equal to zero correspond to actual negatives and predicted negatives, respectively.

To evaluate recovery of β^* itself, we use relative norm $\|\beta^* - \hat{\beta}\|/\|\beta^*\|$. In a replicate, the reported relative norm is the average of the relative norms of each honest client.

7.1.3 Competitors

The data are generated such that each local dataset (X_i, Y_i) contains insufficient samples for a non-federated method (“G”) to perform well in either F_1 or relative norm. By contrast, the global dataset (X, Y) , which is not accessible in the FL setting, contains sufficient samples for “G” to perform well in both F_1 and relative norm. Hence, we include among the competitors two “benchmarks”: “G” (local) and “G” (global). The method “G” (local) uses (non-federated) proximal gradient descent to optimize

$$f_i(\beta) = \frac{1}{100} \|Y_i - X_i \beta\|^2 + \lambda \|\beta\|_1,$$

which depends only on (X_i, Y_i) . The method “G” (global) uses (non-federated) proximal gradient descent to optimize

$$f(\beta) = \frac{1}{1000} \|Y - X \beta\|^2 + \lambda \|\beta\|_1,$$

which depends on the full (X, Y) . Since “G” (local) and “G” (global) are unaffected by the presence of Byzantine clients, the graphs of their performance metrics will be horizontal lines. Generally, we expect that

- In the presence of Byzantine clients, non-Byzantine-tolerant methods will perform worse than “G” (local).
- Up to extreme cases, Byzantine-tolerant methods will outperform “G” (local).
- The performance of “G” (global) is an unattainable upper bound on the performances of the federated methods.

In experiments 1, 2, and 3 we compare the following algorithms:

1. D-SGD
2. D-PSGD
3. “G” (local)
4. ClippedGossip
5. “AG”
6. “A,G”
7. “G,A”
8. “S”
9. “AS”
10. “G” (global)

D-SGD and D-PSGD are non-Byzantine-tolerant methods. D-SGD is the same as “G,A” with clipping and trust scores both turned off, and D-PSGD is the same as “AG” with clipping and trust scores both turned off.

ClippedGossip, “AG”, “A,G”, “G,A”, “S”, and “AS” are Byzantine-tolerant methods. ClippedGossip is the same as “G,A” with trust scores turned off.

7.1.4 Byzantine Clients

We consider three different kinds of Byzantine failures. In all three cases, Byzantine client $i \in \mathcal{B}$ will each be given the same **Byzantine flip proportion** $p_{\text{flip}} \in [0, 1]$, select (uniformly without replacement) p_{flip} proportion of the entries of X_i , Y_i , or g_i , and flip the sign of those entries. Table 3 illustrates these three Byzantine failures.

Byzantine failure	Before flipping	After flipping
X	$X_i = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \end{bmatrix}$	$X_{i,\text{flip}} = \begin{bmatrix} 1 & -2 & -3 & 4 \\ -5 & 6 & -7 & 8 \end{bmatrix}; \quad g_i = \frac{1}{n_i} X_{i,\text{flip}}^\top (X_{i,\text{flip}} \beta - Y_i)$
Y	$Y_i = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$	$Y_{i,\text{flip}} = \begin{bmatrix} -1 \\ 2 \end{bmatrix}; \quad g_i = \frac{1}{n_i} X_i^\top (X_i \beta - Y_{i,\text{flip}})$
g	$g_{ij} = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix}$	$g_i = \begin{bmatrix} 1 \\ -2 \\ -3 \\ 4 \end{bmatrix}$

Table 3: Illustration of 3 different poisoning attacks, where $i \in \mathcal{B}$, $n_i = 2$, $p = 4$, and $p_{\text{flip}} = 1/2$.

It is easily verified that for $p_{\text{flip}} = 0$, all three Byzantine failures are the same (do nothing), and for $p_{\text{flip}} = 1$, “X” and “Y” are the same (but not “g”).

Byzantine clients will always perform “G”, i.e., non-federated proximal gradient descent, but at some point they will poison their data or their gradient. In “X” and “Y”, Byzantine client $i \in \mathcal{B}$ poisons its data once before the first round of the algorithm, obtaining $X_{i,\text{flip}}$ or $Y_{i,\text{flip}}$ respectively, and then compute g_i using their poisoned data. In “g”, Byzantine client $i \in \mathcal{B}$ computes g_i using its unpoisoned data, but then flip entries of g_i directly. In all three cases, g_i winds up poisoned (either directly or being downstream from $X_{i,\text{flip}}$ or $Y_{i,\text{flip}}$), and hence β_i winds up poisoned (being downstream from g_i). Byzantine client i then broadcasts the poisoned $g_{ij} = g_i$ and/or $\beta_{ij} = \beta_i$ to its neighbors $j \in \mathcal{N}_i$.

The setting p_{flip} controls a tradeoff between “unmitigated impact” and “appearing unsuspecting”. For greater values of p_{flip} , an unmitigated Byzantine client can derail convergence to a greater degree, but is also easier to detect and hence mitigate.

Having equated p_{flip} , there is also a difference between “X” and “Y” and “g” with respect to this tradeoff. In “g”, the vector g_{ij} , which is an input to aggregation, is poisoned directly. In “X” and “Y”, g_{ij} winds up poisoned indirectly, being downstream from $X_{i,\text{flip}}$ or $Y_{i,\text{flip}}$. Hence, “g” falls on the “unmitigated impact” side while “X” and “Y” fall on the “appearing unsuspecting” side.

7.1.5 Communication Graph

In experiments 1, 2, and 4, there are $K = 10$ clients; in experiment 3, there are $K = 20$ clients. Experiments 1, 2, and 4 take place on a **mesh**, i.e., complete graph. In experiment 3, the shape of the communication graph varies. Specifically, experiment 3 takes place on a **band**, with $K = 20$, and **connectivity**, denoted by τ , takes values in $\{2, 4, 6, 8, 10\}$. Informally, to form the band, arrange the K nodes in a circle. Then, for each node, draw an edge to the nearest τ nodes on its left and the nearest τ nodes on its right. Figure 4 gives examples of the band with $K = 8$ and varying connectivity.

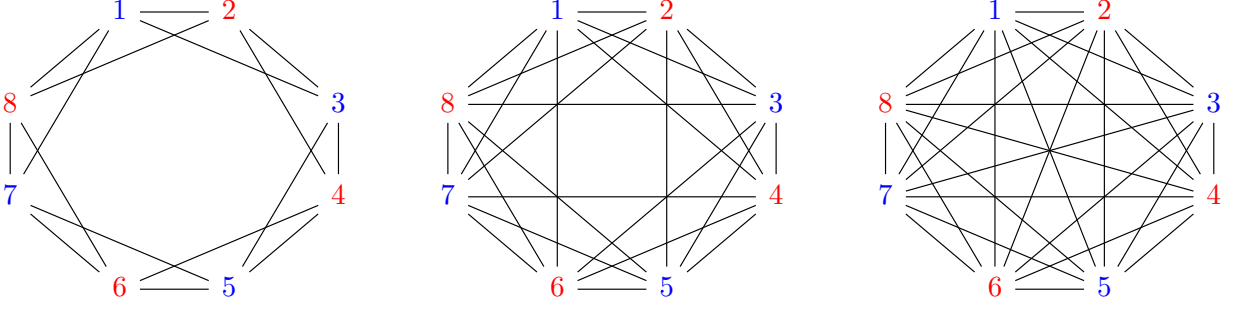


Figure 4: From left to right: the band with $K = 8$ and $\tau = 2$; the band with $K = 8$ and $\tau = 3$; the band with $K = 8$ and $\tau = 4$, equivalently the mesh with $K = 8$. Odd-numbered clients are honest, and even-numbered clients are Byzantine.

In experiment 3, odd-numbered clients are honest and even-numbered clients are Byzantine. This alternating setup ensures that with $\tau \geq 2$, the subgraph induced by honest clients is connected. It also ensures that incrementing connectivity by 2 will give every client four new neighbors, of which two are honest and two are Byzantine. For this reason, τ takes values in $\{2, 4, 6, 8, 10\}$.

7.1.6 Choice of λ

For each experimental setting we report an average over ten replicates. Prior to these ten replicates, we run three replicates with $\lambda \in \{10^{-2}, 10^{-1.5}, 10^{-1}, 10^{-0.5}, 10^0\}$ and record the value of λ maximizing F_1 score and the value of λ minimizing relative norm (which are not necessarily the same). The reported precision, recall, and F_1 scores are obtained with the λ maximizing F_1 and the reported relative norms are obtained with the λ minimizing relative norm.

7.1.7 Additional Details

All clients initialize $\beta_i^0 = 0_{100}$. Step sizes are chosen via golden section search. The number of rounds is set to $M = 100$.

7.2 Results

For the sake of readability, in this subsection we show F_1 scores and relative norms of a subset of the ten listed competitors. Specifically, D-SGD (DFL method that is not Byzantine-tolerant), (benchmark), ClippedGossip, “AS” and “S” (the two best among our methods), and “G” (local) and “G” (global) (non-federated benchmarks). For full results, see appendix B.

7.2.1 Experiment 1

Experiment	K	$ \mathcal{B} $	p_{flip}	$\mathcal{G}$	Byzantine failure
1	10	0 to 9	1	mesh	“Y”
2	10	5	0 to 1	mesh	“Y”
3	20	10	1	band (τ from 2 to 10)	“Y”
4	10	5	0 to 1	mesh	multiple

Table 4: Number of clients, number of Byzantine clients, Byzantine flip proportion, communication graph, and Byzantine failure type for each experiment.

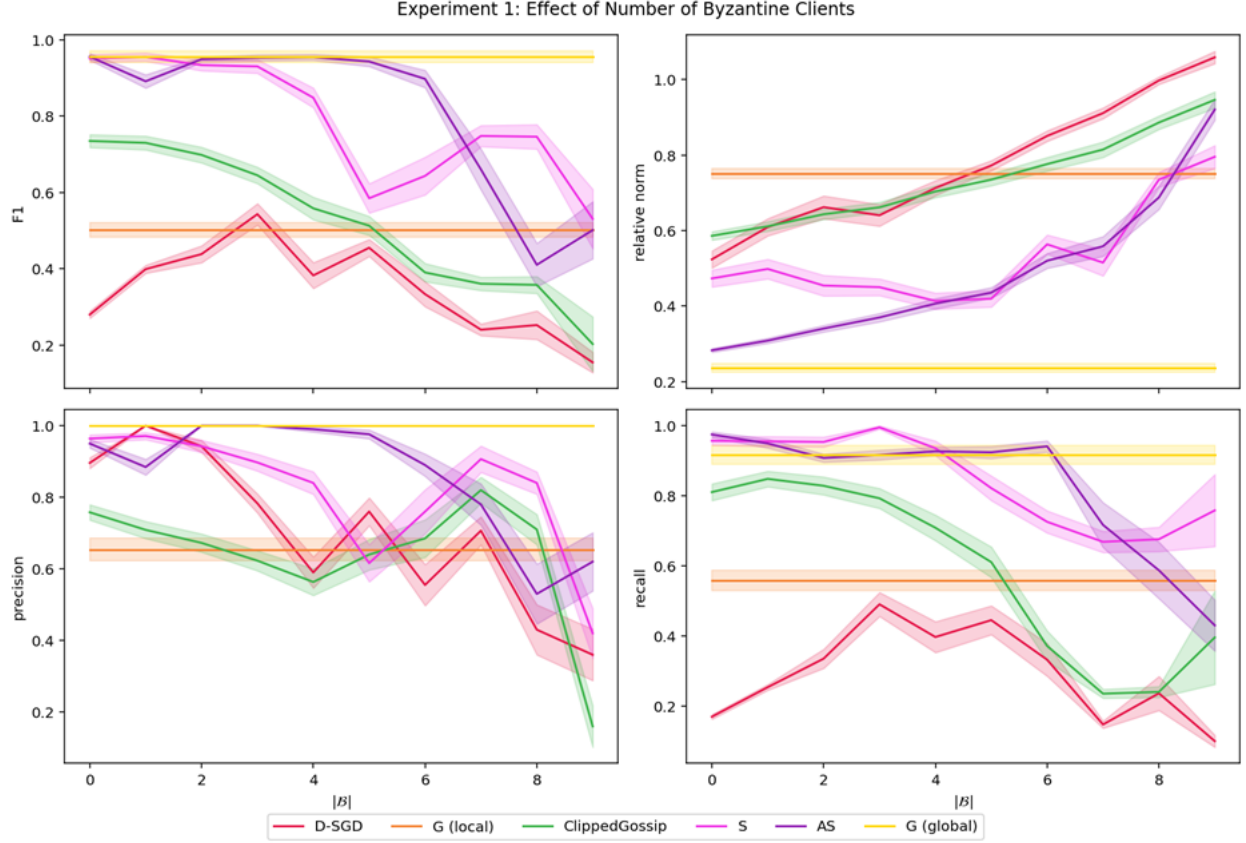


Figure 5: In experiment 1, there are $K = 10$ clients, and the number of Byzantine clients varies in $|\mathcal{B}| = 0, 1, \dots, 9$. Each Byzantine client does “Y”. We observe across methods that performance tends to deteriorate as the number of Byzantine clients increases. In both F_1 and relative norm, ClippedGossip outperforms the non-federated benchmark with fewer than 5 Byzantine clients, and performs worse than the benchmark with more than 5 Byzantine clients. This coheres with subsection 4.2.2, which illustrates that mere clipping can fail against a majority of Byzantine clients. Indeed, $|\mathcal{B}| = 5$ is the inflection point at which Byzantine contributions “match” honest contributions. By contrast, “AS” and “S” make use of trust scores, and hence shrink Byzantine contributions beyond the amount they are already shrunk due to clipping. In both F_1 and relative norm, “AS” and “S” consistently outperform ClippedGossip.

7.2.2 Experiment 2

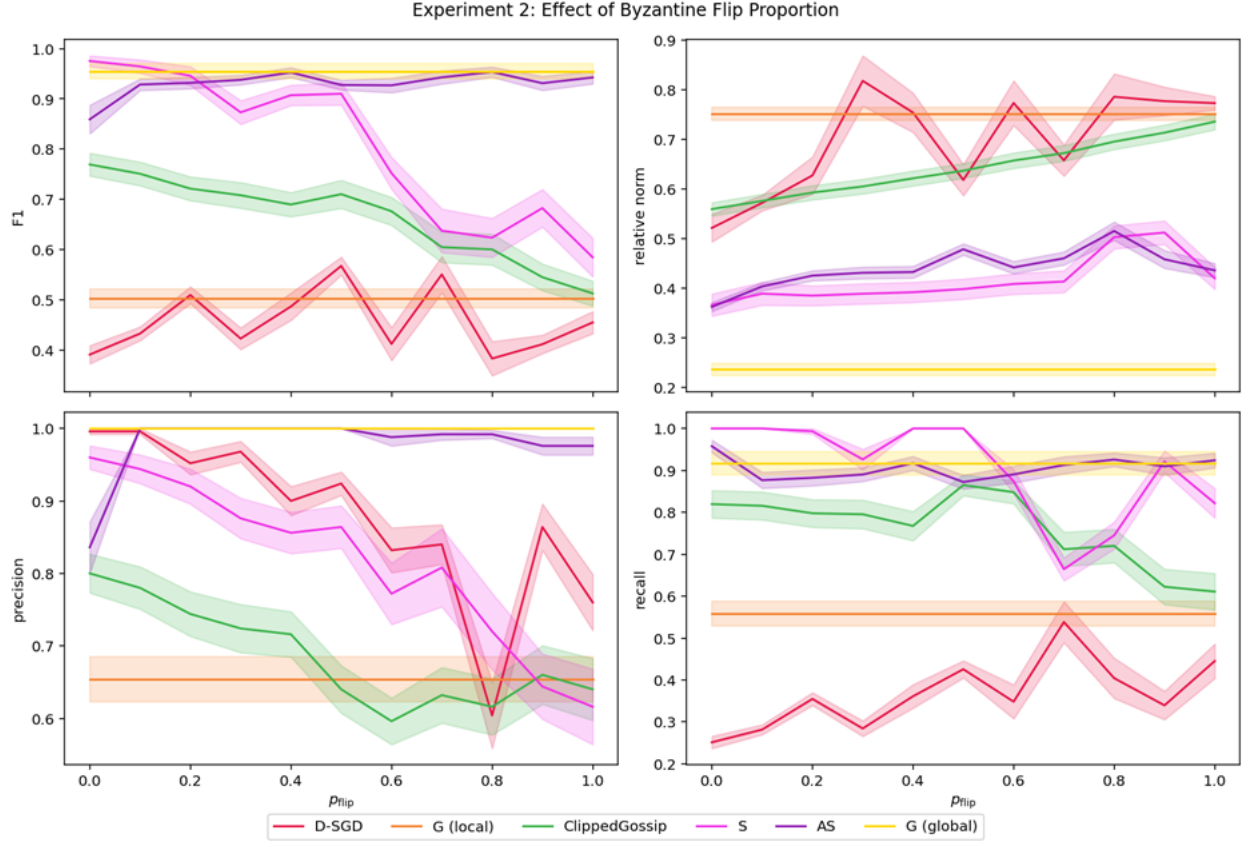


Figure 6: In experiment 2, there are $K = 10$ clients, half of which are Byzantine, which do “Y”. The Byzantine flip proportion varies in $p_{\text{flip}} = 0.0, 0.1, \dots, 0.9, 1.0$. Prior to experiment 2, it was unclear whether performances should improve or deteriorate with p_{flip} . For greater values of p_{flip} , an unmitigated Byzantine client can derail convergence to a greater degree, but is also easier to detect and hence mitigate. While in general methods perform worse as p_{flip} increases, the F_1 score of “AS” improves with p_{flip} , approaching the F_1 score of the gold standard for $p_{\text{flip}} \geq 0.2$. In F_1 and relative norm, “AS” and “S” consistently outperform ClippedGossip.

7.2.3 Experiment 3

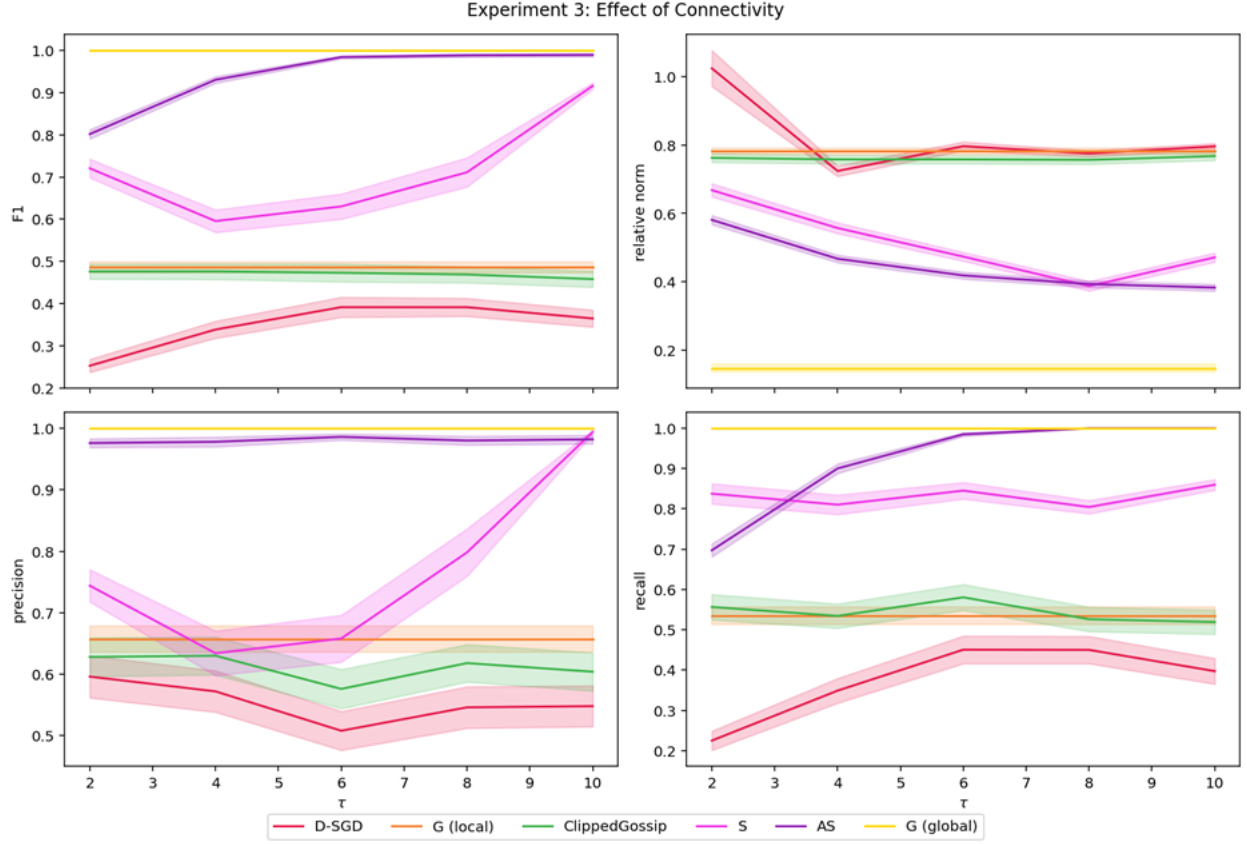


Figure 7: In experiment 3, there are $K = 20$ clients, half of which are Byzantine. Each Byzantine client does “Y”. The communication graph is the band, and connectivity varies in $\tau = 2, 4, 6, 8, 10$. Prior to experiment 3, it was unclear whether performances should improve or deteriorate with connectivity. Incrementing connectivity by 2 gives each client two additional honest neighbors, but also two additional Byzantine neighbors. Figure 8 gives the results of experiment 3. While ClippedGossip appears unaffected by connectivity, “AS” improves as connectivity increases. In both F_1 and relative norm, “AS” and “S” consistently outperform ClippedGossip.

7.2.4 Experiment 4

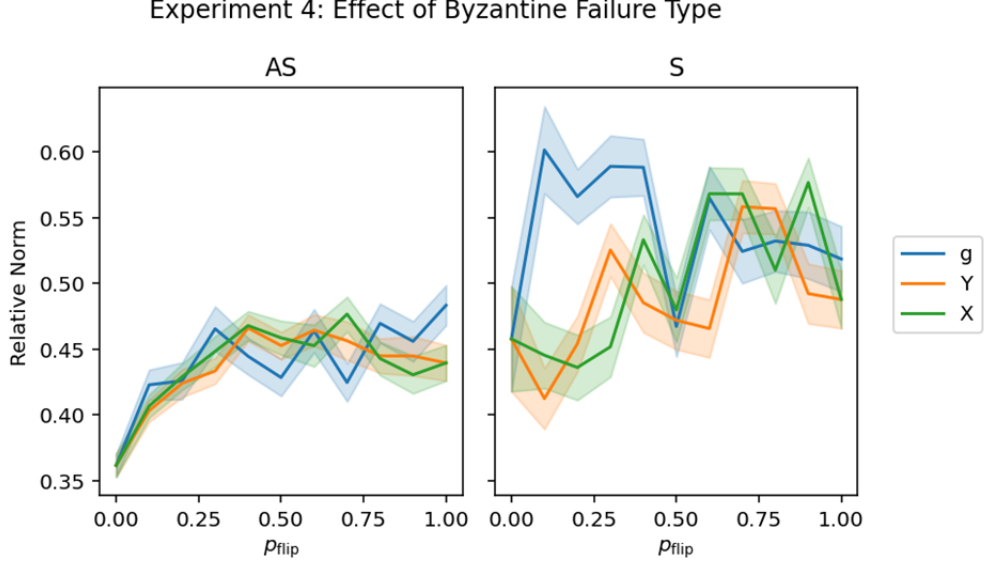


Figure 8: The methods “AS” and “S” perform similarly under different types of Byzantine failure, except for “S” for low values of p_{flip} , for which Byzantine failure “g” is the most effective. One possible reason why is that for low values of p_{flip} , “X” and “Y”, which err on the side of “appearing unsuspecting”, fail to have “unmitigated impact”, whereas “g”, which errs on the side of “unmitigated impact”, properly balances “unmitigated impact” and “appearing unsuspecting”.

8 Future Research Plans

8.1 Generalized Linear Models

Suppose $Y_i \mid X_i$ has a distribution from an exponential family with log-likelihood

$$f = \sum_{\ell=1}^n \frac{Y_{\ell} \theta_{\ell} - b(\theta_{\ell})}{\phi} + c(Y_{\ell}, \phi),$$

where $\theta_{\ell} = (b')^{-1} \circ g^{-1}(X_{\ell}^{\top} \beta) := h(X_{\ell}^{\top} \beta)$. The function g is called the **link function**. We want β maximizing the log-likelihood. This can be found via the **Fisher scoring method**

$$\beta^{m+1} \leftarrow \beta^m + I(\beta^m)^{-1} \nabla f(\beta^m),$$

where I is the Fisher information. This update is equivalent to

$$\beta^{m+1} \leftarrow (X^{\top} W X)^{-1} X^{\top} W (\tilde{Y} - \tilde{\mu} + X \beta^m),$$

where $\tilde{Y} = (g'(\mu_1)Y_1, \dots, g'(\mu_n)Y_n)$, $\tilde{\mu} = (g'(\mu_1)\mu_1, \dots, g'(\mu_n)\mu_n)$, and $W = \text{diag} \left(\frac{h'(X_i^{\top} \beta)}{g'(\mu_i)} \right)$. **Iteratively reweighted least squares (IRLS)** solves the MLE for GLMs using this strategy. The training loop is

1. $\mu_\ell^m \leftarrow g^{-1}(X_\ell^\top \beta^m)$
2. $Z_\ell^m \leftarrow X_\ell^\top \beta^m + g'(\mu_\ell^m)(Y_\ell - \mu_\ell^m)$
3. $W^m \leftarrow \text{diag}\left(\frac{h'(X_\ell^\top \beta^m)}{g'(\mu_\ell^m)\phi}\right)$
4. $\beta^{m+1} \leftarrow (X^\top W^m X)^{-1} X^\top W^m Z^m$.

In [CGA22], IRLS is extended to the CFL setting. Suppose clients $i \in [K]$ each hold X_i, Y_i , with distribution from an exponential family as above. The training loop of **Federated GLM** is

1. Clients $i \in [K]$ do steps 1, 2, and 3 of IRLS locally.
2. Clients $i \in [K]$ broadcast $H_i^m := X_i^\top W_i^m X_i$ and $G_i^m := X_i^\top W_i^m Z_i^m$ to server.
3. Server broadcasts $\beta^{m+1} \leftarrow (\sum_{i=1}^K H_i^m)^{-1} \sum_{i=1}^K G_i^m$ back to clients.

Federated GLM could be generalized to Byzantine DFL by replacing step 3 with

$$\beta_i^{m+1} \leftarrow \left(\sum_{j \in \mathcal{N}_i} t_{ij}^m H_{ij}^m \right)^{-1} \sum_{j \in \mathcal{N}_i} t_{ij}^m G_{ij}^m.$$

Judgments could be computed

$$I_{ij}^{G(m)} = \mathbf{1} \left\{ (G_i^m)^\top G_{ij}^m > 0 \right\}$$

for vectors, and

$$I_{ij}^{H(m)} = \mathbf{1} \left\{ \text{tr}(H_i^m)^\top H_{ij}^m > 0 \right\}$$

for matrices.

8.2 Graphical Models

Let $X = (X_1, \dots, X_p) \in \mathbb{R}^p$ be a random vector with positive density. Let $\mathcal{G}_\perp = ([p], \mathcal{E}_\perp)$ be an undirected graph, where each X_i is identified with the vertex i . The graph $\mathcal{G}_\perp$ is a **conditional independence graph** if for any two vertices $i \neq j \in [p]$,

$$\{i, j\} \notin \mathcal{E}_\perp \implies i \perp j \mid V \setminus \{i, j\}.$$

If additionally X is multivariate normal, we say $\mathcal{G}_\perp$ is a **Gaussian graphical model**. Without loss of generality, we assume $X \sim N_p(0_p, \Omega^{-1})$, where $\Omega \in \mathbb{R}^{p \times p}$ is the **precision matrix**. We are interested in learning the edge set $\mathcal{E}_\perp$ of the graphical model, since it encodes conditional independence relationships between the X_i 's. The next proposition says that learning this edge set is tantamount to learning the support of Ω .

Proposition 1 (adjacency and precision share support). *If $\mathcal{G}_\perp = ([p], \mathcal{E}_\perp)$ is a Gaussian graphical model with respect to $X \sim N_p(0_p, \Omega^{-1})$, then $\{i, j\} \in \mathcal{E}_\perp$ if and only if $\Omega_{ij} \neq 0$.*

The following proposition shows that the precision matrix can be recovered via regression.

Proposition 2 (support of precision as regression coefficients). *Define the matrix $\beta \in \mathbb{R}^{p \times p}$ by $\beta_{ij} = -\Omega_{ij}/\Omega_{jj}$ and $\beta_{ii} = 0$. If $X = [X_1 \dots X_p] \sim N_p(0, \Omega^{-1})$, then X_j given X_{-j} is Gaussian with*

$$\mathbb{E}[X_j \mid X_{-j}] = - \sum_{i \neq j} \beta_{ij} X_i$$

$$\text{Var}(X_j \mid X_{-j}) = \Omega_{jj}^{-1}.$$

We assume that Ω is sparse, otherwise it would not be recoverable in the high-dimensional setting. Proposition 2 along with a sparsity assumption is the setup for p LASSO problems. **Neighborhood regression** [MB06] solves these p LASSO problems as described in Algorithm 6. Let $X \in \mathbb{R}^{n \times p}$, where the rows of X are i.i.d. draws from $N_p(0_p, \Omega^{-1})$.

Algorithm 6 Neighborhood regression

for $\ell \in [p]$ **do**

Solve the LASSO problem $\hat{\beta}_\ell = \arg \min_{\beta \in \mathbb{R}^{p-1}} \left\{ \frac{1}{2n} \|X_{\cdot \ell} - (X_{\cdot -\ell})^\top \beta\|^2 + \lambda \|\beta\|_1 \right\}$

Insert a 0 into $\hat{\beta}_\ell$ at the ℓ -th index. Now $\hat{\beta}^\ell \in \mathbb{R}^p$ and the ℓ -th component is 0.
 $\hat{\beta} \leftarrow [\hat{\beta}_1 \dots \hat{\beta}_p] \in \mathbb{R}^{p \times p}$

return $\hat{A} \in \{0, 1\}^{p \times p}$ defined by $[\hat{A}]_{k\ell} \leftarrow \mathbf{1}\{\hat{\beta}_{k\ell} \neq 0 \text{ AND/OR } \hat{\beta}_{\ell k} \neq 0\}$.

In the Byzantine DFL setting, there are clients $i \in [K]$ each holding $X^{(i)}$, and $X^{(1)}, \dots, X^{(K)} \stackrel{\text{iid}}{\sim} N_p(0_p, \Omega^{-1})$. The K clients want to collaborate to learn Ω without sharing their private datasets $X^{(i)}$. Moreover, some Byzantine clients will introduce Byzantine failures rather than collaborating honestly.

To handle the precision matrix estimation problem in the Byzantine DFL setting, we can run p synchronous instances of the main algorithm, each solving one of the p LASSO problems of neighborhood regression. These p instances are independent except for sharing trust scores. I.e., let $I_{ij}^{\ell m}$ denote client i 's judgment of neighbor j during round m of the ℓ -th LASSO problem. Then, across all p LASSO problems, the trust score client i assigns to neighbor j during round m is

$$t_{ij}^m \leftarrow \frac{1}{mp} \sum_{k=1}^m \sum_{\ell=1}^p I_{ij}^{\ell k}.$$

8.3 Personalized FL

The goal of vanilla FL is to minimize the global objective function

$$f(\beta) = \frac{1}{K} \sum_{i=1}^K f_i(\beta).$$

without sharing local datasets D_i . Clients might be interested in minimizing such an objective function provided:

- We have $D_1, \dots, D_K \stackrel{\text{iid}}{\sim} \mathcal{D}$ for some distribution $\mathcal{D}$.
- Each local dataset D_i does not contain enough samples to recover the optimal model.

In fact, in the proof of Theorem 1, while we do not make any assumptions about data distributions, Assumption 1 (bounded variance) states

$$\mathbb{E} \|g_i^m\| \leq \|\nabla F(\beta_i^m)\|^2 + \sigma^2,$$

which is likelier to hold when $D_1, \dots, D_K \stackrel{\text{iid}}{\sim} \mathcal{D}$.

In **personalized federated learning (PFL)**, the global objective function is

$$f(\beta_1, \dots, \beta_K) = \frac{1}{K} \sum_{i=1}^K f_i(\beta_i).$$

In PFL, each client wants to learn its own “personal” β_i minimizing its local objective function. Clients might be interested in minimizing such an objective function under data heterogeneity, i.e., for $i \in [K]$, $D_i \sim \mathcal{D}_i$, where the $\mathcal{D}_i$ ’s are distinct distributions. If the $\mathcal{D}_i$ ’s are completely arbitrary, then the best solution is for each client to solve its own local problem without any collaboration. So that collaboration is useful, we assume that clients’ data are heterogeneous in some structured way. For example, in the precision matrix estimation setting, we might assume that $X^{(i)} \sim N_p(0_p, \Omega_i^{-1})$, where the Ω_i ’s are distinct. It is believed that the K graphical models share many common edges, but also have some idiosyncratic edges. Figure 9 depicts what the supports of the Ω_i ’s might look like.

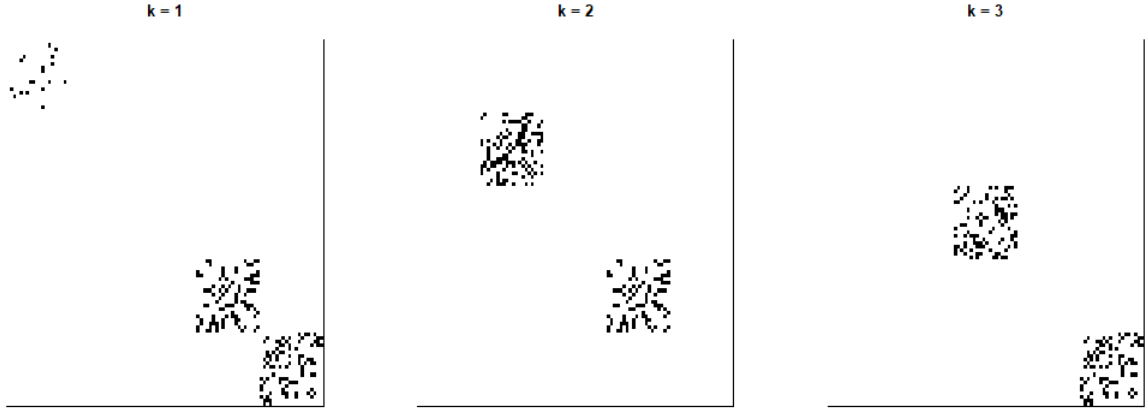


Figure 9: Support of $\Omega_1, \Omega_2, \Omega_3$. In this example, Ω_1 contains some unique edges, and also shares some edges with Ω_2 and other edges with Ω_3 . The supports of Ω_2 and Ω_3 are disjoint. Client 1 would benefit from collaborating with clients 2 and 3, but clients 2 and 3 would not benefit from collaborating with each other.

Since the main algorithm is decentralized and trust-based, it might be suitable for PFL with little or no modification. Since the main algorithm is decentralized, each client learns distinct $\hat{\beta}_i$. In the convergence proof, we show that each β_i , while distinct, converges to the minimizer of $\frac{1}{K} \sum_{i=1}^K f_i(\beta)$. With different assumptions, it might be possible to show that the β_i ’s each converge to the minimizer of their respective local objective function $f_i(\beta)$.

Byzantine failures and heterogeneity appear indistinguishable to a trust-based method. Trust-based methods assume that Byzantine gradients/models are dissimilar to honest gradients/models in some measurable way. A Byzantine client might directly poison its gradient, or it might poison its data, and the gradient winds up poisoned downstream from the poisoned data. If the data is not poisoned but instead drawn from a distinct distribution, there will still be dissimilarity between datasets and hence gradients, which will be caught by a trust-based method. Hence, the more non-identical the data distributions of a client and its neighbor, the more that the client will downweigh the suggestion of that neighbor.

Sometimes, it is assumed that there is prior knowledge regarding the heterogeneity among clients. For example, in [SS16], which takes place in the precision matrix estimation setting, it is assumed that clients $i \in [K]$ correspond to nodes on a **subpopulation graph** $\mathcal{G}_{\text{sim}}([K], \mathcal{E}_{\text{sim}}, W)$, a weighted graph where the weights represent “similarity” between the data distributions of clients. If the precision matrices are those given in Figure 9, then the subpopulation graph would include a weighted edge between clients 1 and 2 and between clients 1 and 3.

The current algorithm can address such prior knowledge using “pseudocounts”. Currently, trust scores are parameterized

$$t_{ij}^m = \frac{\sum_{k=1}^m I_{ij}^k}{m}.$$

We could instead define

$$\begin{aligned} a_{ij}^m &= a_{ij}^0 + \sum_{k=1}^m I_{ij}^k \\ b_{ij}^m &= b_{ij}^0 + \sum_{k=1}^m (1 - I_{ij}^k) \\ t_{ij}^m &= \frac{a_{ij}^m}{a_{ij}^m + b_{ij}^m} = \frac{a_{ij}^0 + \sum_{k=1}^m I_{ij}^k}{a_{ij}^0 + b_{ij}^0 + m}, \end{aligned}$$

where a_{ij}^0 and b_{ij}^0 are called **pseudocounts**. This is equivalent to the current trust score parameterization when $a_{ij}^0 = b_{ij}^0 = 0$. If client i believes it and neighbor j have similar data distributions, client i can initialize a high value of $\frac{a_{ij}^0}{a_{ij}^0 + b_{ij}^0}$. If client i is very certain in its prior belief, it can initialize high values of a_{ij}^0 and b_{ij}^0 , lessening the effect that judgments have on the trust score every round.

A Proof of Convergence

Definition 1 (Objective function). *The **global objective function** is*

$$f(\beta) = F(\beta) + \lambda \|\beta\|_1 = \frac{1}{K} \sum_{i=1}^K F_k(\beta) + \lambda \|\beta\|_1,$$

and the F_i 's (and hence F) are differentiable.

Definition 2 (Optimal model). *Denote by β^* the minimizer of $f(\beta)$.*

Definition 3 (Normalized trust score). ***Normalized trust scores** are defined by $w_{ij}^m = \frac{t_{ij}^m}{\sum_{j \in \mathcal{N}_i} t_{ij}^m}$ and sum to 1.*

Definition 4 (Aggregated suggestion). *Under model update sequence “ S ”, the **aggregated suggestion** is*

$$G_i^m = \sum_{j \in \mathcal{N}_i} w_{ij}^m \tilde{g}_{ij}^m.$$

Assumption 1 (Bounded variance). *The local stochastic gradient satisfies*

$$\mathbb{E} \|g_i^m\|^2 \leq \|\nabla F(\beta_i^m)\|^2 + \sigma^2.$$

This implies

$$\mathbb{E} \|g_i^m\| \leq \|\nabla F(\beta_i^m)\| + \sigma.$$

Assumption 2 (Behavior of neighbors).

1. For $j \in \mathcal{H}_i$ the judgments $I_{ij}^1, \dots, I_{ij}^M$ are i.i.d. from $\text{Bern}(0.5 + \delta)$ for some $\delta > 0$.
2. For $j \in \mathcal{B}_i$ and $m \in [M]$, we have $\mathbb{P}(I_{ij}^m = 1) \leq 0.5$.

Assumption 3 (Honest alignment). *For $j \in \mathcal{H}_i$,*

$$\mathbb{E} \langle \tilde{g}_{ij}^m, \nabla F(\beta_i^m) \rangle \geq C \|\nabla F(\beta_i^m)\|.$$

Assumption 4 (L -smoothness). *For all $\beta, \beta' \in \mathbb{R}^p$,*

$$\|\nabla F(\beta) - \nabla F(\beta')\| \leq L \|\beta - \beta'\|.$$

Assumption 5 (Bounded gradient). *The norm of the local gradient is bounded by*

$$\|\nabla F(\beta)\| \leq D := L \|\beta^0 - \beta^*\|.$$

Assumption 6 (Bounded Byzantine ratio). *The Byzantine ratio is bounded by*

$$\frac{|\mathcal{B}_i|}{|\mathcal{H}_i|} \leq \frac{(0.5 + \delta)C}{0.5(D + \sigma)}.$$

Assumption 7 (μ -strong convexity). *For all $\beta, \beta' \in \mathbb{R}^p$,*

$$\langle \beta - \beta', \nabla F(\beta) - \nabla F(\beta') \rangle \geq \mu \|\beta - \beta'\|^2.$$

Lemma 1 (Clipping bound). *For all $j \in \mathcal{N}_i$,*

$$\|\tilde{g}_{ij}^m\| \leq \|g_i^m\|.$$

Proof. We have

$$\|\tilde{g}_{ij}^m\| = \left\| g_{ij}^m \min \left\{ 1, \frac{\|g_i^m\|}{\|g_{ij}^m\|} \right\} \right\| = \|g_{ij}^m\| \min \left\{ 1, \frac{\|g_i^m\|}{\|g_{ij}^m\|} \right\} = \min \{ \|g_{ij}^m\|, \|g_i^m\| \} \leq \|g_i^m\|.$$

□

Lemma 2 (Aggregation preserves bounded variance via clipping). *Under Assumption 1, the aggregated suggestion satisfies*

$$\mathbb{E} \|G_i^m\|^2 \leq \mathbb{E} \|g_i^m\|^2 \leq \|\nabla F(\beta_i^m)\|^2 + \sigma^2.$$

Proof. We have

$$\begin{aligned} \|G_i^m\| &= \left\| \sum_{j \in \mathcal{N}_i} w_{ij}^m \tilde{g}_{ij}^m \right\| && \text{(Definition of aggregated suggestion)} \\ &\leq \sum_{j \in \mathcal{N}_i} w_{ij}^m \|\tilde{g}_{ij}^m\| && \text{(Triangle inequality)} \\ &\leq \left(\sum_{j \in \mathcal{N}_i} w_{ij}^m \right) \|g_i^m\| && \text{(Lemma 1 (clipping bound))} \\ &= \|g_i^m\| && \text{(Normalized trust scores sum to 1)} \\ &\implies \|G_i^m\|^2 \leq \|g_i^m\|^2 && \text{(Norms are non-negative)} \\ &\implies \mathbb{E} \|G_i^m\|^2 \leq \mathbb{E} \|g_i^m\|^2 \\ &\implies \mathbb{E} \|G_i^m\|^2 \leq \mathbb{E} \|g_i^m\|^2 \leq \|\nabla F(\beta_i^m)\|^2 + \sigma^2. && \text{(Assumption 1 (bounded variance))} \end{aligned}$$

□

Lemma 3 (Hoeffding's inequality). *Suppose $X_1, \dots, X_n$ are independent Bernoulli random variables, and $S_n = X_1 + \dots + X_n$. Then*

$$\mathbb{P} \left(\frac{S_n - \mathbb{E} S_n}{n} \geq t \right) \leq \exp(-2nt^2)$$

for all $t \geq 0$.

Lemma 4 (Borel-Cantelli). *If the sum of the probabilities of the events $\{E_n\}$ is finite then the probability that infinitely many of them occur is 0.*

Lemma 5 (Asymptotic trust bounds). *Let $i \in \mathcal{H}$, and let $I_{ij}^m \in \{0, 1\}$ be client i 's judgment of neighbor $j \in \mathcal{N}_i$ at iteration m . The trust score at iteration k is $t_{ij}^k = \frac{1}{k} \sum_{m=1}^k I_{ij}^m$. For neighbor $j \in \mathcal{N}_i$, put $p_{ij}^m := \mathbb{P}(I_{ij}^m = 1)$. Under Assumption 2,*

1. If for $m \in \mathbb{N}$, $p_{ij}^m = 0.5 + \delta$, then $t_{ij}^k \xrightarrow{a.s.} 0.5 + \delta$.
2. If for $k \in \mathbb{N}$, $p_{ij}^k \leq 0.5$, then $\limsup_{k \rightarrow \infty} t_{ij}^k \leq 0.5$ almost surely.

Proof. Step 1: Honest trust scores. This follows from Assumption 2 and the strong law of large numbers.

Step 2: Byzantine trust scores. The expected trust score at round k is $\mathbb{E} t_{ij}^k = \frac{1}{k} \sum_{m=1}^k p_{ij}^m \leq 0.5$, where the inequality holds by Assumption 2. Then, by Hoeffding's inequality,

$$\begin{aligned} \mathbb{P}\left(t_{ij}^k - \mathbb{E} t_{ij}^k \geq \epsilon\right) &\leq \exp(-2k\epsilon^2) && \text{(Lemma 3 (Hoeffding's inequality))} \\ \implies \mathbb{P}\left(t_{ij}^k - 0.5 \geq \epsilon\right) &\leq \exp(-2k\epsilon^2). && (\mathbb{E} t_{ij}^k \leq 0.5) \end{aligned}$$

The series $\sum_{k=1}^{\infty} \exp(-2k\epsilon^2)$ is convergent for $\epsilon > 0$. By Lemma 4 (Borel-Cantelli), the probability that the event $\{t_{ij}^k \geq 0.5 + \epsilon\}$ occurs for infinitely many terms is 0. If $\limsup t_{ij}^k > 0.5$ almost surely, then $\limsup t_{ij}^k = 0.5 + \epsilon$ for some $\epsilon > 0$ almost surely. But then $t_{ij} \geq 0.5 + \epsilon$ for infinitely many terms almost surely, so it must be the case that $\limsup t_{ij}^k \leq 0.5$ almost surely. $\square$

Lemma 6 (Expected descent direction). *Under Assumptions 1, 2, 3, 5, and 6,*

$$\langle \mathbb{E} G_i^m, \nabla F(\beta_i^m) \rangle \geq 0.$$

Proof. Step 1: Expand the inner product. We have

$$\langle \mathbb{E} G_i^m, \nabla F(\beta_i^m) \rangle = \sum_{j \in \mathcal{H}_i} w_{ij}^m \mathbb{E} \langle \tilde{g}_{ij}^m, \nabla F(\beta_i^m) \rangle + \sum_{j \in \mathcal{B}_i} w_{ij}^m \mathbb{E} \langle \tilde{g}_{ij}^m, \nabla F(\beta_i^m) \rangle.$$

Step 2: Bound the Byzantine term. We have

$$\begin{aligned} \sum_{j \in \mathcal{B}_i} w_{ij}^m \mathbb{E} \langle \tilde{g}_{ij}^m, \nabla F(\beta_i^m) \rangle &\geq - \sum_{j \in \mathcal{B}_i} w_{ij}^m \mathbb{E} \|\tilde{g}_{ij}^m\| \|\nabla F(\beta_i^m)\| && \text{(Cauchy-Schwarz)} \\ &\geq - \left(\sum_{j \in \mathcal{B}_i} w_{ij}^m \right) (\|\nabla F(\beta_i^m)\| + \sigma) \|\nabla F(\beta_i^m)\|. \\ &&& \text{(Assumption 1 (bounded variance))} \end{aligned}$$

Step 3: Bound the honest term. By Assumption 3 (honest alignment),

$$\sum_{j \in \mathcal{H}_i} w_{ij}^m \mathbb{E} \langle \tilde{g}_{ij}^m, \nabla F(\beta_i^m) \rangle \geq \left(\sum_{j \in \mathcal{H}_i} w_{ij}^m \right) C \|\nabla F(\beta_i^m)\|.$$

Step 4: Put it together. Putting together steps 1, 2, and 3, we have

$$\langle \mathbb{E} G_i^m, \nabla F(\beta_i^m) \rangle \geq \left(\sum_{j \in \mathcal{H}_i} w_{ij}^m \right) C \|\nabla F(\beta_i^m)\| - \left(\sum_{j \in \mathcal{B}_i} w_{ij}^m \right) (\|\nabla F(\beta_i^m)\| + \sigma) \|\nabla F(\beta_i^m)\|.$$

By Lemma 5 (asymptotic trust bounds), we have asymptotically that

$$\langle \mathbb{E} G_i^m, \nabla F(\beta_i^m) \rangle \geq \frac{1}{T} |\mathcal{H}_i| (0.5 + \delta) C \|\nabla F(\beta_i^m)\| - \frac{1}{T} |\mathcal{B}_i| (0.5) (\|\nabla F(\beta_i^m)\| + \sigma) \|\nabla F(\beta_i^m)\|,$$

Then,

$$\begin{aligned} \langle \mathbb{E} G_i^m, \nabla F(\beta_i^m) \rangle &\geq \frac{1}{T} |\mathcal{H}_i| (0.5 + \delta) C \|\nabla F(\beta_i^m)\| - \frac{1}{T} |\mathcal{B}_i| (0.5) (\|\nabla F(\beta_i^m)\| + \sigma) \|\nabla F(\beta_i^m)\| \\ &= \frac{\|\nabla F(\beta_i^m)\|}{T} [|\mathcal{H}_i| (0.5 + \delta) C - |\mathcal{B}_i| (0.5) (\|\nabla F(\beta_i^m)\| + \sigma)] \\ &\geq \frac{\|\nabla F(\beta_i^m)\|}{T} [|\mathcal{H}_i| (0.5 + \delta) C - |\mathcal{B}_i| (0.5) (D + \sigma)] \\ &\quad \text{(Assumption 5 (bounded gradient))} \\ &\geq \frac{\|\nabla F(\beta_i^m)\|}{T} \left[|\mathcal{H}_i| (0.5 + \delta) C - \frac{|\mathcal{H}_i| (0.5 + \delta) C}{0.5(D + \sigma)} (0.5)(D + \sigma) \right] \\ &\quad \text{(Assumption 6 (bounded Byzantine ratio))} \\ &= 0. \end{aligned}$$

□

Lemma 7 (Optimal model as fixed point). *For any $\eta > 0$, the optimal model satisfies*

$$\beta^* = S_{\eta\lambda}(\beta^* - \eta \nabla F(\beta^*)).$$

Lemma 8 (Non-expansiveness of the soft-thresholding operator). *For all $\beta, \beta' \in \mathbb{R}^p$,*

$$\|S_{\eta\lambda}(\beta) - S_{\eta\lambda}(\beta')\| \leq \|\beta - \beta'\|.$$

Lemma 9. *For $x_1, \dots, x_n \in \mathbb{R}^p$,*

$$\left\| \sum_{i=1}^n x_i \right\|^2 \leq n \sum_{i=1}^n \|x_i\|^2.$$

Theorem 1 (Convergence). *Under Assumptions 1, 2, 3, 4, 5, 6, and 7, and with constant step size $0 < \eta < \min \left\{ \frac{\mu}{4L^2}, \frac{2}{\mu} \right\}$,*

$$\mathbb{E} \|\beta_i^m - \beta^*\|^2 \leq \left(1 - \frac{\eta\mu}{2}\right)^m \|\beta_i^0 - \beta^*\|^2 + \frac{10}{\mu^2} (2L^2 \|\beta^0 - \beta^*\|^2 + \sigma^2).$$

Proof. Step 1: Expand $\|\beta^{m+1} - \beta^\|^2$.* Lemma 7 says

$$\beta^* = S_{\eta\lambda}(\beta^* - \eta \nabla F(\beta^*)).$$

The update is

$$\beta_i^{m+1} = S_{\eta\lambda}(\beta_i^m - \eta G_i^m).$$

Hence,

$$\begin{aligned}
\|\beta_i^{m+1} - \beta^*\|^2 &= \|S_{\eta\lambda}(\beta_i^m - \eta G_i^m) - S_{\eta\lambda}(\beta^* - \eta \nabla F(\beta^*))\|^2 \\
&\leq \|\beta_i^m - \eta G_i^m - \beta^* + \eta \nabla F(\beta^*)\|^2 \quad (\text{Lemma 8 (non-expansiveness)}) \\
&= \|\beta_i^m - \beta^*\|^2 - 2\eta \langle \beta_i^m - \beta^*, G_i^m - \nabla F(\beta^*) \rangle + \eta^2 \|G_i^m - \nabla F(\beta^*)\|^2.
\end{aligned}$$

Step 2: Bound $\langle \beta_i^m - \beta^*, G_i^m - \nabla F(\beta^*) \rangle$. Define $\Delta_i^m = G_i^m - \nabla F(\beta_i^m)$, so

$$\begin{aligned}
\langle \beta_i^m - \beta^*, G_i^m - \nabla F(\beta^*) \rangle &= \langle \beta_i^m - \beta^*, \Delta_i^m + \nabla F(\beta_i^m) - \nabla F(\beta^*) \rangle \\
&= \langle \beta_i^m - \beta^*, \Delta_i^m \rangle + \langle \beta_i^m - \beta^*, \nabla F(\beta_i^m) - \nabla F(\beta^*) \rangle \\
&\geq \langle \beta_i^m - \beta^*, \Delta_i^m \rangle + \mu \|\beta_i^m - \beta^*\|^2 \quad (\text{Assumption 7 } (\mu\text{-strong convexity})) \\
&\geq \mu \|\beta_i^m - \beta^*\|^2 - \|\beta_i^m - \beta^*\| \|\Delta_i^m\| \quad (\text{Cauchy-Schwarz}) \\
&= \mu \|\beta_i^m - \beta^*\|^2 - \sqrt{\mu} \|\beta_i^m - \beta^*\| \frac{1}{\sqrt{\mu}} \|\Delta_i^m\| \\
&\geq \mu \|\beta_i^m - \beta^*\|^2 - \frac{\mu \|\beta_i^m - \beta^*\|^2}{2} - \frac{\|\Delta_i^m\|^2}{2\mu} \quad (\text{Young's inequality}) \\
&= \frac{\mu \|\beta_i^m - \beta^*\|^2}{2} - \frac{\|\Delta_i^m\|^2}{2\mu}.
\end{aligned}$$

Step 3: Bound $\|G_i^m - \nabla F(\beta^*)\|^2$. We have

$$\begin{aligned}
\|G_i^m - \nabla F(\beta^*)\|^2 &= \|G_i^m - \nabla F(\beta_i^m) + \nabla F(\beta_i^m) - \nabla F(\beta^*)\|^2 \\
&\leq 2\|\Delta_i^m\|^2 + 2\|\nabla F(\beta_i^m) - \nabla F(\beta^*)\|^2 \quad (\text{Lemma 9}) \\
&\leq 2\|\Delta_i^m\|^2 + 2L^2 \|\beta_i^m - \beta^*\|^2. \quad (\text{Assumption 4 } (L\text{-smoothness}))
\end{aligned}$$

Step 4: Put it together. Putting Steps 1, 2, and 3 together, we have

$$\begin{aligned}
\|\beta_i^{m+1} - \beta^*\|^2 &\leq \|\beta_i^m - \beta^*\|^2 - 2\eta \left(\frac{\mu \|\beta_i^m - \beta^*\|^2}{2} - \frac{\|\Delta_i^m\|^2}{2\mu} \right) + 2\eta^2 (\|\Delta_i^m\|^2 + L^2 \|\beta_i^m - \beta^*\|^2) \\
&= (1 - \eta\mu + 2\eta^2 L^2) \|\beta_i^m - \beta^*\|^2 + \left(\frac{\eta}{\mu} + 2\eta^2 \right) \|\Delta_i^m\|^2 \\
&\leq \left(1 - \frac{\eta\mu}{2} \right) \|\beta_i^m - \beta^*\|^2 + \left(\frac{\eta}{\mu} + 2\eta^2 \right) \|\Delta_i^m\|^2. \quad (\eta < \frac{\mu}{4L^2})
\end{aligned}$$

Step 5: Unroll the geometric sequence. Unrolling the geometric sequence over m steps yields

$$\|\beta_i^m - \beta^*\|^2 \leq \left(1 - \frac{\eta\mu}{2} \right)^m \|\beta^0 - \beta^*\|^2 + \left(\frac{\eta}{\mu} + 2\eta^2 \right) \|\Delta_i^m\|^2 \sum_{k=0}^{m-1} \left(1 - \frac{\eta\mu}{2} \right)^k$$

By assumption, the constant step size satisfies $\eta < \frac{2}{\mu}$, ensuring that the common ratio $(1 - \frac{\eta\mu}{2})$ is between 0 and 1. The sum of the geometric series is

$$\sum_{k=0}^{\infty} \left(1 - \frac{\eta\mu}{2} \right)^k = \frac{1}{1 - (1 - \frac{\eta\mu}{2})} = \frac{2}{\eta\mu}.$$

Hence,

$$\begin{aligned}
\|\beta_i^m - \beta^*\|^2 &\leq \left(1 - \frac{\eta\mu}{2}\right)^m \|\beta^0 - \beta^*\|^2 + \left(\frac{\eta}{\mu} + 2\eta^2\right) \|\Delta_i^m\|^2 \frac{2}{\eta\mu} \\
&= \left(1 - \frac{\eta\mu}{2}\right)^m \|\beta^0 - \beta^*\|^2 + \left(\frac{2}{\mu^2} + 4\frac{\eta}{\mu}\right) \|\Delta_i^m\|^2 \\
&\leq \left(1 - \frac{\eta\mu}{2}\right)^m \|\beta^0 - \beta^*\|^2 + \frac{10}{\mu^2} \|\Delta_i^m\|^2 \quad (\eta < \frac{2}{\mu}) \\
\implies \mathbb{E} \|\beta_i^m - \beta^*\|^2 &\leq \left(1 - \frac{\eta\mu}{2}\right)^m \|\beta^0 - \beta^*\|^2 + \frac{10}{\mu^2} \mathbb{E} \|\Delta_i^m\|^2.
\end{aligned}$$

Step 6: Bound $\mathbb{E} \|\Delta_i^m\|^2$. We have

$$\begin{aligned}
\mathbb{E} \|\Delta_i^m\|^2 &= \mathbb{E} \|G_i^m - \nabla F(\beta_i^m)\|^2 \\
&= \mathbb{E} \|G_i^m\|^2 - 2\langle \mathbb{E} G_i^m, \nabla F(\beta_i^m) \rangle + \|\nabla F(\beta_i^m)\|^2 \\
&\leq 2\|\nabla F(\beta_i^m)\|^2 + \sigma^2 - 2\langle \mathbb{E} G_i^m, \nabla F(\beta_i^m) \rangle \quad (\text{Lemma 2}) \\
&\leq 2L^2\|\beta^0 - \beta^*\|^2 + \sigma^2 - 2\langle \mathbb{E} G_i^m, \nabla F(\beta_i^m) \rangle \quad (\text{Assumption 5 (bounded gradient)}) \\
&\leq 2L^2\|\beta^0 - \beta^*\|^2 + \sigma^2. \quad (\text{Lemma 6 (expected descent direction)})
\end{aligned}$$

Step 7: Final result. We have

$$\mathbb{E} \|\beta_i^m - \beta^*\|^2 \leq \left(1 - \frac{\eta\mu}{2}\right)^m \|\beta^0 - \beta^*\|^2 + \frac{10}{\mu^2} (2L^2\|\beta^0 - \beta^*\|^2 + \sigma^2).$$

□

B Full Numerical Results

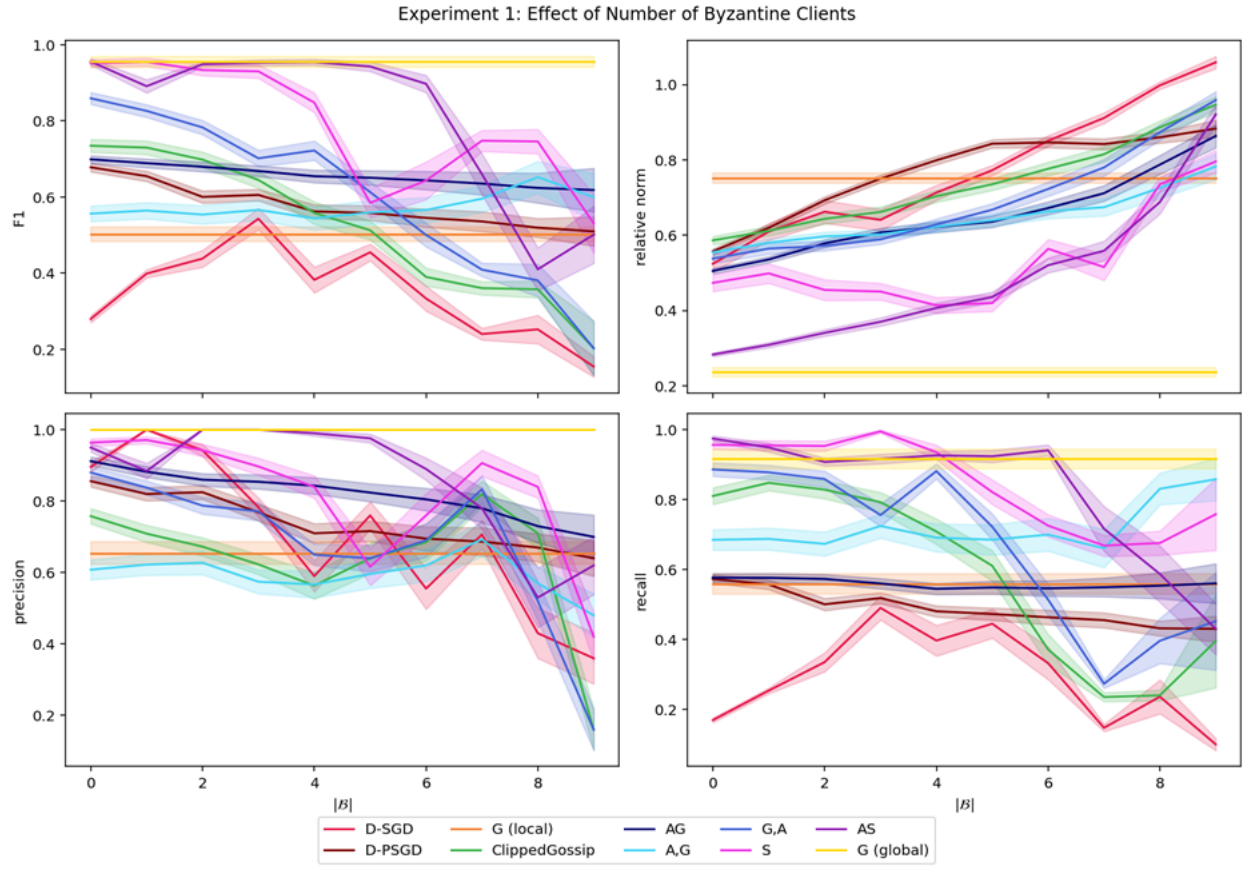


Figure 10: Full results of experiment 1.

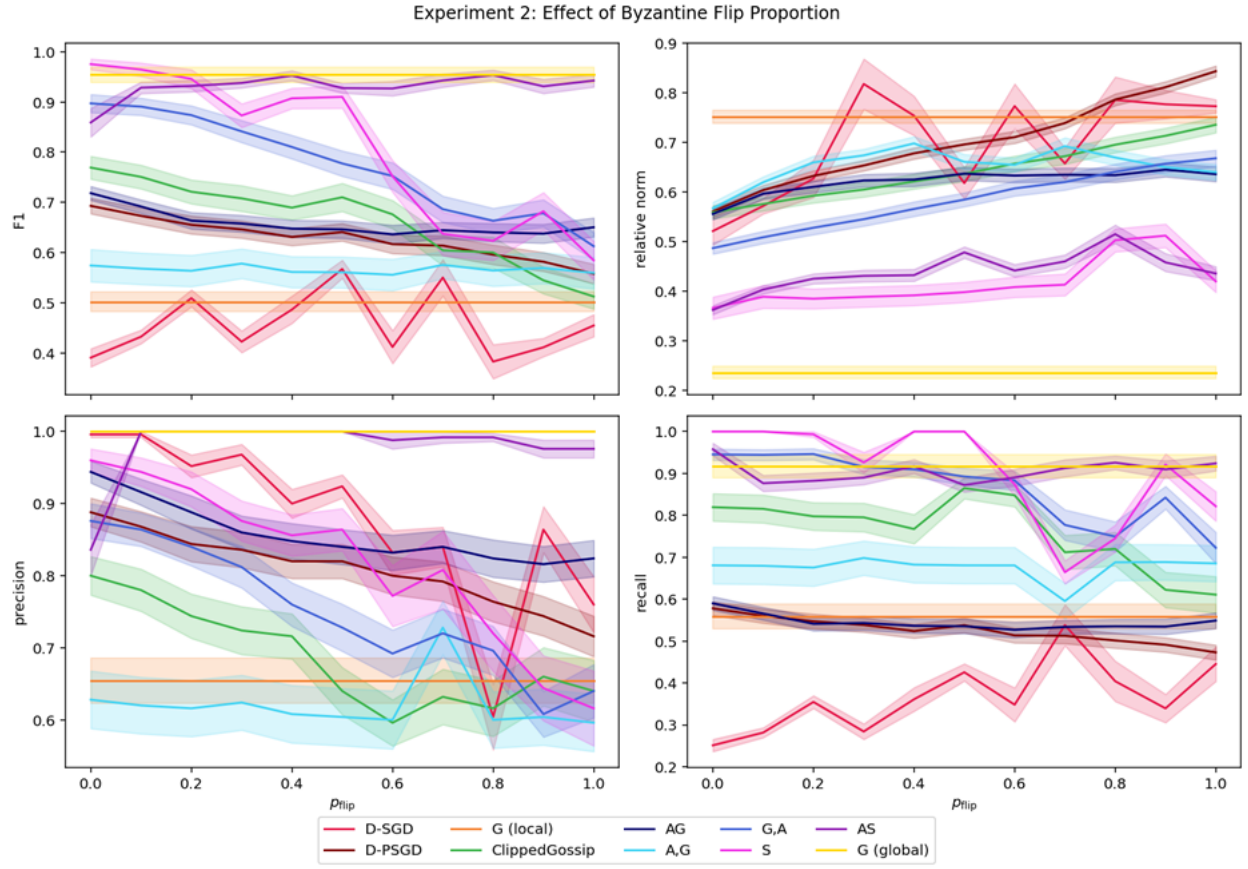


Figure 11: Full results of experiment 2.

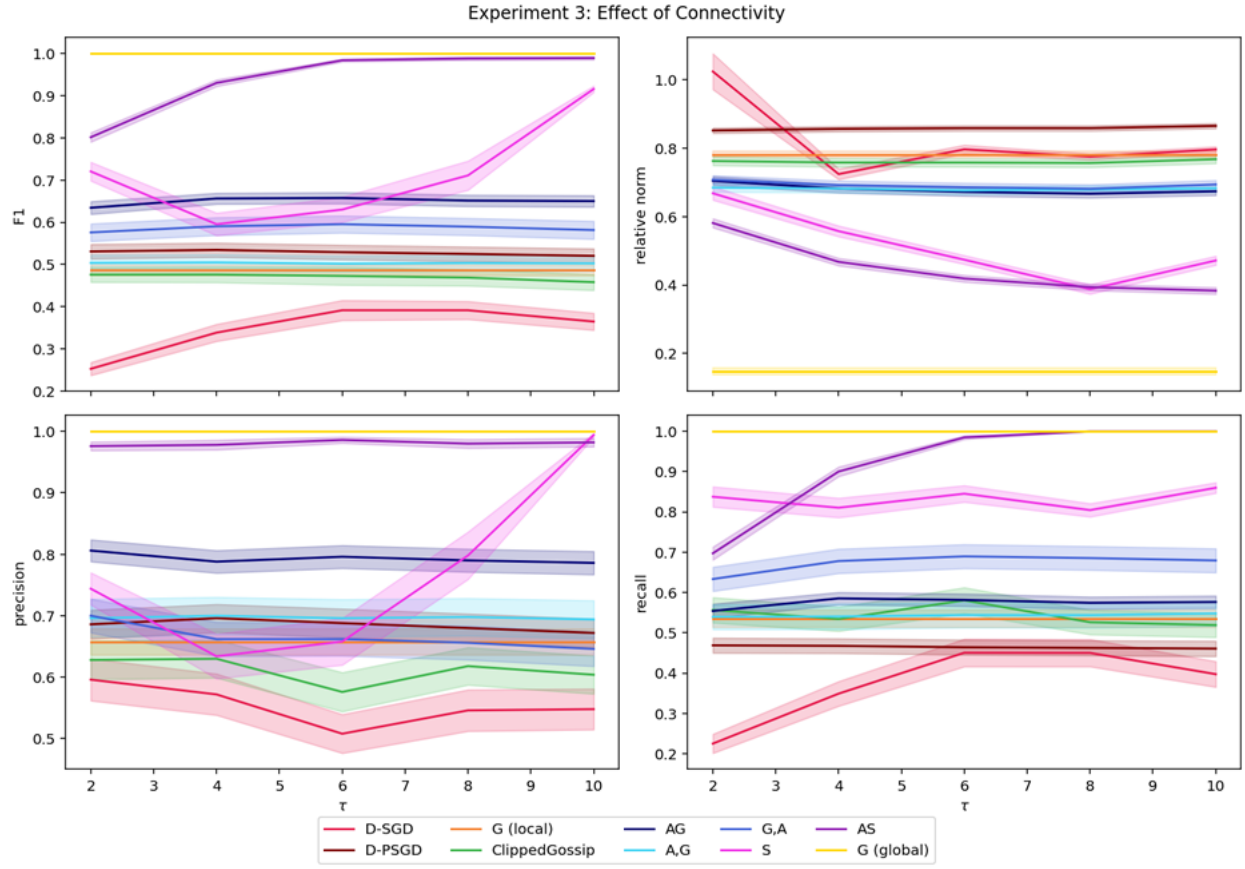


Figure 12: Full results of experiment 3.

References

- [BEG17] Peva Blanchard, El Mahdi El Mhamdi, Rachid Guerraoui, and Julien Stainer. “Machine Learning with Adversaries: Byzantine Tolerant Gradient Descent.” In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.
- [CCL19] Di Cao, Shan Chang, Zhijian Lin, Guohua Liu, and Donghong Sun. “Understanding Distributed Poisoning Attack in Federated Learning.” In *2019 IEEE 25th International Conference on Parallel and Distributed Systems (ICPADS)*, pp. 233–239, 2019.
- [CFL20] Xiaoyu Cao, Minghong Fang, Jia Liu, and Neil Zhenqiang Gong. “FLTrust: Byzantine-robust Federated Learning via Trust Bootstrapping.” *CoRR*, **abs/2012.13995**, 2020.
- [CGA22] Matteo Cellamare, Anna J. van Gestel, Hasan Alradhi, Frank Martin, and Arturo Moncada-Torres. “A Federated Generalized Linear Model for Privacy-Preserving Analysis.” *Algorithms*, **15**(7), 2022.
- [GTB22] Anousheh Gholami, Nariman Torkzaban, and John S. Baras. “Trusted Decentralized Federated Learning.” In *2022 IEEE 19th Annual Consumer Communications & Networking Conference (CCNC)*, pp. 1–6, 2022.
- [HKJ23] Lie He, Sai Praneeth Karimireddy, and Martin Jaggi. “Byzantine-Robust Decentralized Learning via ClippedGossip.”, 2023.
- [KLB21] Anastasia Koloskova, Nicolas Loizou, Sadra Boreiri, Martin Jaggi, and Sebastian U. Stich. “A Unified Theory of Decentralized SGD with Changing Topology and Local Updates.”, 2021.
- [LLX21] Xiaoyuan Liu, Hongwei Li, Guowen Xu, Zongqi Chen, Xiaoming Huang, and Rongxing Lu. “Privacy-Enhanced Federated Learning Against Poisoning Adversaries.” *IEEE Transactions on Information Forensics and Security*, **16**:4574–4588, 2021.
- [LSP82] Leslie Lamport, Robert Shostak, and Marshall Pease. “The Byzantine generals problem.”, Jul 1982.
- [LWW21] Dongcheng Li, W. Eric Wong, Wei Wang, Yao Yao, and Matthew Chau. “Detection and Mitigation of Label-Flipping Attacks in Federated Learning Systems with KPCA and K-Means.” In *2021 8th International Conference on Dependable Systems and Their Applications (DSA)*, pp. 551–559, 2021.
- [LZZ17] Xiangru Lian, Ce Zhang, Huan Zhang, Cho-Jui Hsieh, Wei Zhang, and Ji Liu. “Can Decentralized Algorithms Outperform Centralized Algorithms? A Case Study for Decentralized Parallel Stochastic Gradient Descent.”, 2017.
- [MB06] Nicolai Meinshausen and Peter Bühlmann. “High-dimensional graphs and variable selection with the Lasso.” *The Annals of Statistics*, **34**(3), 2006.

- [MMR16] H. Brendan McMahan, Eider Moore, Daniel Ramage, and Blaise Agüera y Arcas. “Federated Learning of Deep Networks using Model Averaging.” *CoRR*, **abs/1602.05629**, 2016.
- [PB14] Neal Parikh and Stephen Boyd. “Proximal Algorithms.” *Found. Trends Optim.*, **1**(3):127–239, January 2014.
- [SS16] Takumi Saegusa and Ali Shojaie. “Joint estimation of precision matrices in heterogeneous populations.” *Electronic Journal of Statistics*, **10**(1):1341 – 1392, 2016.
- [YCR18] Dong Yin, Yudong Chen, Kannan Ramchandran, and Peter L. Bartlett. “Byzantine-Robust Distributed Learning: Towards Optimal Statistical Rates.” *CoRR*, **abs/1803.01498**, 2018.
- [ZW24] Xudong Zhang and Lei Wang. “High-dimensional M-estimation for Byzantine-robust decentralized learning.” *Information Sciences*, **653**:119808, 2024.